

INFORMATIQUE • 1ÈRE C

MODULE 2 : ALGORITHMIQUE ET PROGRAMMATION

LEÇON N07

ALGORITHMIQUE : STRUCTURES DE CONTRÔLE, FONCTIONS ET PROCÉDURES

DURÉE CONSEILLÉE : 9 H

DANS CETTE LEÇON

Cette leçon achève la boîte à outils algorithmique de Première : structures itératives, tableaux à une dimension, fonctions et procédures avec passage de paramètres. L'élève apprend à concevoir des algorithmes modulaires et efficaces résolvant des problèmes concrets de mathématiques et de physique.

- À la fin de cette leçon, je sais utiliser les structures itératives (Pour, TantQue, Répéter) dans un algorithme
- À la fin de cette leçon, je sais écrire des algorithmes résolvant des problèmes simples de mathématiques et de physique combinant structures séquentielles, conditionnelles et itératives
- À la fin de cette leçon, je sais énumérer quelques structures de données : enregistrements, tableaux, piles, listes
- À la fin de cette leçon, je sais déclarer un tableau à une dimension et le parcourir (lecture, affichage, recherche)
- À la fin de cette leçon, je sais exécuter à la main un algorithme simple utilisant les tableaux
- À la fin de cette leçon, je sais déclarer une fonction et une procédure et les distinguer

**TÉLÉCHARGE ONBUCH**

Gratuit • Google Play
Tuteur IA, annales,
concours, résultats

**PASSE À ONBUCH+**

Tous les cours signés, Tuteur IA
illimité, fiches PDF — onglet
OnBuch+ de l'app

CE COURS SIGNÉ CONTIENT

COURS

complet, illustré, pas à pas

MÉTHODES

et exercices résolus

EXERCICES

type examen + corrigés

FICHE BILAN

l'essentiel à retenir

**Signé • OnBuch+**

Ludovic A.
CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+

SOMMAIRE

- | | |
|--|----------------------------------|
| 1. Les structures itératives | bales |
| 2. Algorithmes sur les problèmes de mathématiques et de physique | 7. Activité d'intégration |
| 3. Les structures de données : panorama | 8. Méthodes et exercices résolus |
| 4. Le type tableau à une dimension | 9. Exercices |
| 5. Fonctions et procédures | 10. Corrigés des exercices |
| 6. Paramètres, variables locales et glo- | 11. Fiche bilan |

OBJECTIFS DE LA LEÇON

- À la fin de cette leçon, je sais utiliser les structures itératives (Pour, TantQue, Répéter) dans un algorithme
- À la fin de cette leçon, je sais écrire des algorithmes résolvant des problèmes simples de mathématiques et de physique combinant structures séquentielles, conditionnelles et itératives
- À la fin de cette leçon, je sais énumérer quelques structures de données : enregistrements, tableaux, piles, listes
- À la fin de cette leçon, je sais déclarer un tableau à une dimension et le parcourir (lecture, affichage, recherche)
- À la fin de cette leçon, je sais exécuter à la main un algorithme simple utilisant les tableaux
- À la fin de cette leçon, je sais déclarer une fonction et une procédure et les distinguer
- À la fin de cette leçon, je sais identifier le mode de passage d'un paramètre (par valeur ou par référence)
- À la fin de cette leçon, je sais distinguer une variable locale d'une variable globale
- À la fin de cette leçon, je sais exécuter pas à pas un algorithme intégrant une fonction ou une procédure

PRÉREQUIS

- Notion d'algorithme, de variable et de type de données (leçons précédentes du mo-

dule 2)

- Structure séquentielle et structure conditionnelle Si...Alors...Sinon
- Opérateurs arithmétiques (mod, div) et opérateurs logiques
- Notions de mathématiques : suites, sommes, divisibilité, pgcd
- Notions de physique de Seconde/Première : mouvement uniforme, loi d'Ohm (pour les applications)

1 LES STRUCTURES ITÉRATIVES

EXEMPLE Situation d'accroche : la cybercafétéria de Yaoundé

À la cybercafétéria du quartier Nlongkak à Yaoundé, le gérant accueille chaque jour une cinquantaine de clients. Chaque client paie selon sa durée de connexion : 300 FCFA l'heure, avec une réduction de 10 % au-delà de 2 heures. Le soir, le gérant veut connaître la recette totale et le client le plus fidèle de la journée.

Faire les calculs à la main, client après client, est long et source d'erreurs. Écrire le même calcul 50 fois dans un programme serait absurde. Trois questions se posent alors naturellement :

- Comment répéter un calcul 50 fois sans l'écrire 50 fois ?
- Comment mémoriser les 50 montants ?
- Comment réutiliser le même calcul de facture dans plusieurs parties du programme ?

La première question trouve sa réponse dans les **structures itératives**, objet de cette section. Les deux autres seront traitées avec les tableaux, puis les fonctions et procédures.

1.1 Rappel : structures séquentielle et conditionnelle

Avant de répéter, rappelons ce que nous savons déjà faire.

DÉFINITION Structures déjà connues

Un algorithme est une suite finie et ordonnée d'**instructions** permettant de résoudre un problème.

- La **structure séquentielle** : les instructions s'exécutent les unes après les autres, dans l'ordre d'écriture (lecture, calcul, affichage).
- La **structure conditionnelle** (Si ... Alors ... Sinon ... FinSi) : une instruction n'est exécutée que si une condition est vraie.

EXEMPLE Réactivation : facture d'un seul client

Algorithme FactureClient

Variables duree, montant : Reel

Debut

Ecrire("Durée de connexion (en heures) : ")

```
Lire(duree)
montant <- duree * 300
Si duree > 2 Alors
    montant <- montant * 0.9 // reduction de 10 %
FinSi
Ecrire("Montant a payer :", montant, " FCFA")
Fin
```

Ce programme traite un seul client. Pour 50 clients, il faudrait le recopier 50 fois : inacceptable. Il nous manque un outil pour dire à la machine : « refais ce bloc d'instructions tant que nécessaire ».

1.2 Notion de structure itérative

DÉFINITION Structure itérative (boucle) et compteur

Une **structure itérative** (ou **boucle**) est une structure de contrôle qui permet d'exécuter plusieurs fois un même bloc d'instructions, appelé corps de la boucle. Chaque exécution du corps s'appelle une **itération**.

Un **compteur** est une variable entière dont la valeur évolue (généralement de 1 en 1) à chaque itération et qui permet de contrôler le nombre de répétitions.

Il existe trois formes de boucles en algorithmique : Pour, TantQue et Répéter. Elles se distinguent par la façon dont on décide de s'arrêter.

1.3 La boucle Pour ... de ... à ... Faire

Intuition. Le gérant sait à l'avance qu'il a exactement 50 clients à traiter. Quand le nombre de répétitions est connu à l'avance, la boucle Pour est l'outil naturel : le compteur part d'une valeur initiale et progresse automatiquement jusqu'à une valeur finale.

PROPRIÉTÉ Syntaxe et fonctionnement de la boucle Pour

```
Pour compteur de valeurInitiale a valeurFinale Faire
    instructions
FinPour
```

- À l'entrée, le compteur reçoit la `valeurInitiale`.
- À la fin de chaque itération, le compteur est automatiquement incrémenté de 1 (on dit que le pas vaut 1).
- La boucle s'arrête dès que le compteur dépasse la `valeurFinale`.

- Le nombre d'itérations vaut donc : $\text{valeurFinale} - \text{valeurInitiale} + 1$.

ATTENTION Ne jamais modifier le compteur dans le corps d'un Pour

À l'intérieur d'une boucle `Pour`, il est interdit d'écrire une instruction qui modifie le compteur (par exemple `i <- i + 2`) : le comportement deviendrait imprévisible. Le compteur est géré automatiquement par la boucle.

EXEMPLE Exemple chiffré : la somme $1 + 2 + \dots + 100$

On veut calculer $S = 1 + 2 + 3 + \dots + 100$. L'idée : une variable `somme`, initialisée à 0, à laquelle on ajoute tour à tour chaque valeur du compteur.

```

Algorithme Somme100
Variables i, somme : Entier
Debut
    somme <- 0      // initialisation indispensable
    Pour i de 1 à 100 Faire
        somme <- somme + i // on ajoute i à l'accumulateur
    FinPour
    Ecrire("La somme vaut : ", somme) // affiche 5050
Fin
  
```

La variable `somme` joue le rôle d'**accumulateur** : elle « accumule » les valeurs au fil des itérations. Après la dernière itération, elle contient bien 5050.

LESAVAIS-TU ? Le jeune Gauss et la somme des 100 premiers entiers

Vers 1787, l'instituteur du jeune Carl Friedrich Gauss, alors âgé d'environ 10 ans, demanda à sa classe de calculer $1 + 2 + \dots + 100$, espérant occuper ses élèves un long moment. Gauss donna la réponse, 5050, presque instantanément ! Il avait remarqué qu'en regroupant les termes deux à deux ($1 + 100 = 101$, $2 + 99 = 101$, ...), on obtient 50 paires valant chacune 101, d'où $50 \times 101 = 5050$. C'est la formule générale :

$$1 + 2 + \dots + n = \frac{n(n+1)}{2}.$$

Moralité : une boucle fait le travail en n itérations, alors qu'une bonne formule mathématique le fait en une seule opération. L'algorithmique et les mathématiques se complètent : réfléchir avant de programmer fait gagner du temps !

1.4 La boucle TantQue ... Faire

Intuition. Revenons à la cybercafétéria : le gérant ne sait pas toujours à l'avance combien de clients viendront. Il veut saisir les durées de connexion tant qu'il reste des clients. Quand le nombre de répétitions est inconnu à l'avance, on utilise une boucle conditionnelle : `TantQue`.

PROPRIÉTÉ Syntaxe et fonctionnement de la boucle TantQue

```
TantQue condition Faire
    instructions
FinTantQue
```

- La condition est évaluée avant chaque itération (condition d'entrée).
- Si la condition est vraie, le corps de la boucle s'exécute, puis on reteste la condition.
- Si la condition est fausse dès le départ, le corps n'est jamais exécuté : la boucle peut donc s'exécuter 0 fois.
- Le nombre d'itérations n'est pas connu à l'avance : il dépend des données.

EXEMPLE Saisie des durées tant qu'il reste des clients

```
Algorithme RecetteJournee
Variables duree, montant, recette : Reel
Variable reponse : Chaine
Debut
    recette <- 0
    Ecrire("Reste-t-il un client ? (oui/non) : ")
    Lire(reponse)
    TantQue reponse = "oui" Faire
        Ecrire("Durée de connexion : ")
        Lire(duree)
        montant <- duree * 300
        Si duree > 2 Alors
            montant <- montant * 0.9
        FinSi
        recette <- recette + montant
        Ecrire("Reste-t-il un client ? (oui/non) : ")
        Lire(reponse) // indispensable : fait évoluer la condition
    FinTantQue
    Ecrire("Recette totale : ", recette, " FCFA")
Fin
```

Remarque la structure typique : une lecture d'amorce avant la boucle, puis une nouvelle lecture à la fin du corps pour faire évoluer la condition.

ATTENTION Erreur fréquente : la boucle infinie

L'erreur la plus classique avec `TantQue` est d'oublier de faire évoluer la condition dans le corps de la boucle. Exemple fautif :

```
i <- 1
TantQue i <= 10 Faire
    Ecrire(i)
    // ERREUR : on a oublié i <- i + 1
FinTantQue
```

Ici, `i` vaut toujours 1 : la condition reste vraie indéfiniment et le programme ne s'arrête jamais (on dit qu'il plante ou qu'il boucle). Règle d'or : toute boucle `TantQue` doit contenir au moins une instruction qui modifie une variable intervenant dans la condition, de façon à la rendre fausse un jour.

1.5 La boucle Répéter ... Jusqu'à

Intuition. Parfois, on est certain de devoir exécuter le corps **au moins une fois** : par exemple, demander un mot de passe — on doit le saisir au moins une fois avant de pouvoir le vérifier. La boucle `Répéter` teste sa condition après le corps.

PROPRIÉTÉ Syntaxe et fonctionnement de la boucle `Répéter`

```
Repeter
    instructions
Jusqu'à condition
```

- Le corps s'exécute d'abord, puis la condition est évaluée.
- Si la condition est fausse, on recommence ; si elle est vraie, on sort de la boucle. C'est donc une condition de sortie (logique inverse du `TantQue`).
- Le corps est exécuté au moins une fois, quoi qu'il arrive.

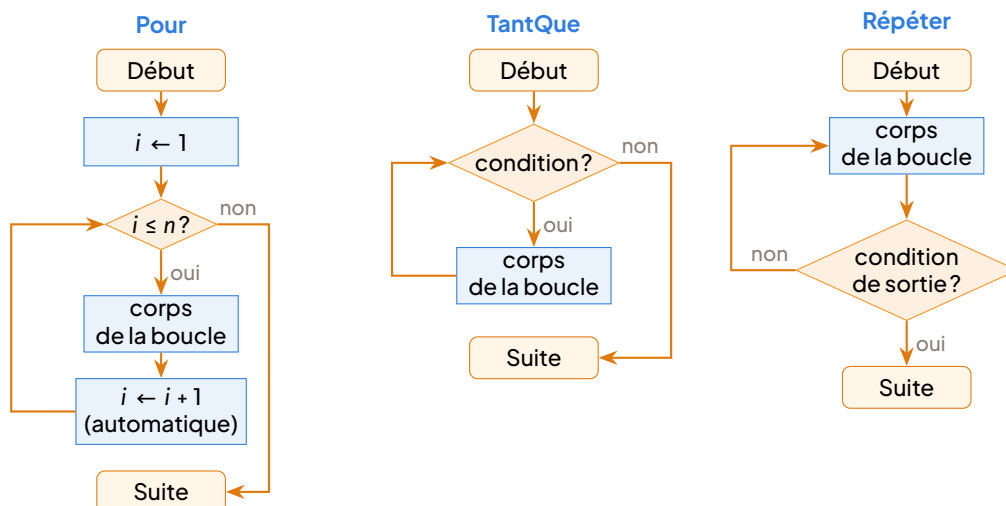
EXEMPLE Saisie contrôlée d'une durée positive

```
Repeter
    Ecrire("Entrez la durée de connexion (en heures, > 0) : ")
    Lire(duree)
Jusqu'à duree > 0
```

Même si l'utilisateur se trompe trois fois de suite (valeur négative ou nulle), la saisie est

redemandée jusqu'à obtenir une valeur valide. Avec un `TantQue`, il aurait fallu initialiser artificiellement `duree` avant la boucle.

1.6 Comparaison des trois boucles et critères de choix



Organigrammes des trois boucles : dans `Pour` et `TantQue`, le test est fait avant le corps; dans `Répéter`, il est fait après (le corps s'exécute donc au moins une fois).

À RETENIR Quelle boucle choisir ?

- Nombre d'itérations connu à l'avance (traiter 50 clients, calculer $n!$, afficher une table) : boucle `Pour`.
- Nombre d'itérations inconnu, corps pouvant ne jamais s'exécuter (tant qu'il reste des clients, tant que le solde est suffisant) : boucle `TantQue`.
- Corps devant s'exécuter au moins une fois (saisie contrôlée, menu) : boucle `Répéter`.

PROPRIÉTÉ Du Pour vers le TantQue (admise)

Toute boucle `Pour` peut se réécrire avec un `TantQue` : il suffit d'initialiser le compteur avant la boucle et de l'incrémenter explicitement à la fin du corps.

// Boucle Pour	// Reécriture en TantQue
Pour i de 1 à n Faire	$i \leftarrow 1$

instructions	TantQue $i \leq n$ Faire
FinPour	instructions
	$i \leftarrow i + 1$
	FinTantQue

En revanche, la réciproque est fautive en général : un **TantQue** dont le nombre d'itérations dépend des données saisies (par exemple « tant qu'il reste des clients ») ne peut pas s'écrire avec un **Pour**, puisque le **Pour** exige de connaître à l'avance les bornes du compteur. On le vérifiera par trace d'exécution en exercice.

1.7 La trace d'exécution : méthode systématique

Pour vérifier qu'un algorithme est correct — ou pour comprendre ce qu'il fait —, on le « fait tourner à la main » : c'est la **trace d'exécution**.

MÉTHODE Construire une trace d'exécution pas à pas

1. Repérer les variables de l'algorithme et créer une colonne par variable, plus une colonne « itération » (numéro de passage dans la boucle).
2. Écrire les valeurs initiales (ligne 0, avant la boucle).
3. Exécuter mentalement chaque itération, dans l'ordre : pour chaque passage, recopier sur une nouvelle ligne la valeur de chaque variable après exécution du corps de la boucle.
4. Vérifier la condition d'arrêt : la boucle s'arrête-t-elle au bon moment ? Le nombre de lignes correspond-il au nombre d'itérations attendu ?
5. Contrôler le résultat final affiché : est-il cohérent avec ce qu'on attendait ?

EXERCICE RÉSOLU 1 Trace de la factorielle de 5

Énoncé. On considère l'algorithme suivant, qui calcule $n! = 1 \times 2 \times \dots \times n$ pour $n = 5$:

Algorithme Factorielle
Variables i , n , $fact$: Entier
Debut
$n \leftarrow 5$
$fact \leftarrow 1$
Pour i de 1 à n Faire
$fact \leftarrow fact * i$
FinPour
Ecrire(n , " ! = ", $fact$)

Fin

Construire la trace d'exécution et donner la valeur affichée.

SOLUTION

Solution détaillée. Les variables sont `i` (compteur) et `fact` (accumulateur, initialisé à 1 car on multiplie). La boucle s'exécute pour `i` allant de 1 à 5, soit 5 itérations :

Itération	<code>i</code>	Calcul effectué	<code>fact</code>
0 (avant boucle)	-	initialisation	1
1	1	1×1	1
2	2	1×2	2
3	3	2×3	6
4	4	6×4	24
5	5	24×5	120

À la sortie de la boucle, `fact` vaut 120. Le programme affiche donc : $5! = 120$. C'est bien la valeur attendue puisque $5! = 120$. La trace confirme la correction de l'algorithme.

ATTENTION Initialiser l'accumulateur avec la bonne valeur

Pour une somme, l'accumulateur part de 0 (élément neutre de l'addition). Pour un produit (factorielle, puissance), il part de 1 (élément neutre de la multiplication). Initialiser `fact` à 0 donnerait $0 \times 1 \times 2 \times \dots = 0$: résultat faux alors que la boucle est correcte !

1.8 Applications classiques

Table de multiplication. Afficher la table de 7, pour un élève de sixième qui révise :

```

Algorithme TableDe7
Variables i, produit : Entier
Debut
    Pour i de 1 a 10 Faire
        produit <- 7 * i
        Ecrire("7 x ", i, " = ", produit)
    FinPour
Fin
  
```

Moyenne de n notes. Un professeur de Première C au lycée de Nkongsamba veut la moyenne des notes d'informatique d'une classe. Le nombre n d'élèves est connu (il est saisi), donc on utilise un `Pour` :

```

Algorithme MoyenneNotes
  
```

```

Variables i, n : Entier
Variables note, somme, moyenne : Reel
Debut
  Ecrire("Nombre d'eleves : ")
  Lire(n)
  somme <- 0
  Pour i de 1 a n Faire
    Ecrire("Note de l'eleve ", i, " : ")
    Lire(note)
    somme <- somme + note
  FinPour
  moyenne <- somme / n
  Ecrire("Moyenne de la classe : ", moyenne)
Fin

```

Application physique : mouvement rectiligne uniforme. En physique, un mobile en mouvement rectiligne uniforme a pour équation horaire $x(t) = v \cdot t + x_0$, où v est la vitesse (en m/s) et x_0 la position initiale (en m). On veut afficher la position du mobile chaque seconde, pour t de 0 à 10 s, avec $v = 5$ m/s et $x_0 = 20$ m :

```

Algorithme PositionsMobile
Variables t : Entier
Variables v, x0, x : Reel
Debut
  v <- 5
  x0 <- 20
  Pour t de 0 a 10 Faire
    x <- v * t + x0
    Ecrire("t = ", t, " s   x = ", x, " m")
  FinPour
Fin

```

La boucle s'exécute 11 fois ($10 - 0 + 1$) et affiche successivement $x = 20, 25, 30, \dots, 70$ m : on retrouve le tableau de valeurs dressé en cours de physique, obtenu ici automatiquement.

EXERCICE 1 À toi de jouer

1. Écrire un algorithme qui calcule et affiche la somme des carrés des n premiers entiers : $1^2 + 2^2 + \dots + n^2$, n étant saisi au clavier.
2. Réécrire l'algorithme `Factorielle` en utilisant une boucle `TantQue`, puis construire sa trace pour $n = 4$.
3. Un mobile part de $x_0 = 0$ avec une vitesse $v = 10$ m/s. Écrire un algorithme qui détermine (avec un `TantQue`) le premier instant t entier (en secondes) où sa position dépasse 100 m.

CORRIGÉ Exercice 1

```
Algorithme SommeCarres
Variables i, n, somme : Entier
Debut
  Ecrire("Entrez n : ")
  Lire(n)
  somme <- 0
  Pour i de 1 a n Faire
    somme <- somme + i * i
  FinPour
  Ecrire("Somme des carres : ", somme)
Fin
```

CORRIGÉ Exercice 2

```
Algorithme FactorielleTQ
Variables i, n, fact : Entier
Debut
  n <- 4
  fact <- 1
  i <- 1
  TantQue i <= n Faire
    fact <- fact * i
    i <- i + 1 // ne pas oublier l'incrementation !
  FinTantQue
  Ecrire(n, "! = ", fact) // affiche 4 ! = 24
Fin
```

Trace : (i, fact) vaut successivement $(1,1), (2,2), (3,6), (4,24)$; quand i devient 5, la condition $i \leq 4$ est fausse et on sort avec $\text{fact} = 24 = 4!$.

CORRIGÉ Exercice 3

```
Algorithme InstantDepassement
Variables t : Entier
Variables x : Reel
Debut
  t <- 0
  x <- 0
  TantQue x <= 100 Faire
```

```
t <- t + 1
x <- 10 * t    // x(t) = v.t avec x0 = 0
FinTantQue
Ecrire("Le mobile dépasse 100 m à t = ", t, " s")
Fin
```

La boucle s'arrête dès que $x > 100$, c'est-à-dire pour $t = 11$ s (car $x(10) = 100$ n'est pas un dépassement strict). Le nombre d'itérations n'était pas connu à l'avance : le `TantQue` s'imposait.

À RETENIR L'essentiel de la section

- Une **boucle** répète un bloc d'instructions ; chaque passage est une **itération**.
- **Pour** : nombre d'itérations connu, compteur géré automatiquement.
- **TantQue** : test avant le corps, 0 itération possible ; penser à faire évoluer la condition sous peine de boucle infinie.
- **Répéter ... Jusqu'à** : test après le corps, au moins une itération.
- Tout **Pour** se réécrit en **TantQue**, mais pas l'inverse en général.
- La **trace d'exécution** (tableau de suivi des variables) est l'outil systématique pour vérifier un algorithme.

2 ALGORITHMES SUR LES PROBLÈMES DE MATHÉMATIQUES ET DE PHYSIQUE

Dans la section précédente, tu as découvert les structures itératives : la boucle **Pour** (nombre de répétitions connu), la boucle **TantQue** (répétition conditionnée) et la boucle **Répéter... Jusqu'à**. Tu sais donc désormais combiner les trois grandes familles de structures de contrôle : séquentielle, conditionnelle et itérative. Mais savoir manier un marteau ne suffit pas pour construire une maison : encore faut-il savoir quel outil utiliser, quand et pourquoi. C'est précisément l'objet de cette section : apprendre à **résoudre des problèmes concrets** de mathématiques et de physique en choisissant et en assemblant correctement les structures algorithmiques. Chaque problème sera traité selon une démarche rigoureuse et reproductible, que tu pourras réutiliser au Probatoire comme dans tes études supérieures.

2.1 La démarche générale de résolution d'un problème

Face à un énoncé (« écrire un algorithme qui... »), l'erreur la plus fréquente consiste à se jeter immédiatement sur l'écriture du code. Or un algorithme se construit, comme on construit un plan avant de bâtir une maison à Yaoundé ou à Douala. La démarche professionnelle comporte

quatre étapes.

MÉTHODE Les 4 étapes de résolution algorithmique d'un problème

1. Comprendre le problème. Lis attentivement l'énoncé, reformule-le avec tes propres mots et teste-le mentalement sur un exemple numérique simple. Pose-toi la question : « que doit faire exactement mon algorithme ? »
2. Découper : identifier les entrées, les sorties et les traitements. Les **entrées** sont les données fournies à l'algorithme (ce qu'on lui donne) ; les **sorties** sont les résultats qu'il doit produire (ce qu'il rend) ; le **traitement** est la suite d'actions qui transforme les entrées en sorties. C'est à cette étape que tu choisis les structures de contrôle : une simple suite d'instructions (séquentielle), un choix (conditionnelle `Si...Alors...Sinon`), une répétition (boucle `Pour` ou `TantQue`) ?
3. Coder l'algorithme en pseudo-code rigoureux : déclaration des variables avec leurs types, initialisations, puis corps du traitement, en respectant la syntaxe exacte.
4. Tracer l'algorithme : exécute-le « à la main » sur un jeu de données test, en construisant un **tableau d'exécution** (une colonne par variable, une ligne par étape). C'est le seul moyen fiable de détecter les erreurs de logique avant toute programmation.

À RETENIR Le réflexe « entrées – traitement – sorties »

Tout algorithme, même complexe, se ramène à trois questions : qu'est-ce qu'on me donne ? (entrées), qu'est-ce qu'on me demande ? (sorties), comment passer de l'un à l'autre ? (traitement). Cette analyse systématique est le premier savoir-faire exigé au Probatoire.

2.2 Problème 1 : tester si un entier est premier

Analyse. Un entier naturel $n \geq 2$ est dit **premier** s'il n'admet aucun diviseur autre que 1 et lui-même. Pour tester la primalité de n , l'idée naïve consiste à essayer de diviser n par tous les entiers d de 2 à $n-1$: si l'un d'eux divise n (c'est-à-dire si le reste de la division euclidienne de n par d vaut 0), alors n n'est pas premier. Une optimisation classique : il suffit de tester les diviseurs d tels que $d \leq \sqrt{n}$, car si $n = a \times b$ avec $a \leq b$, alors $a \leq \sqrt{n}$.

Entrées / sorties. Entrée : un entier n . Sortie : un message « premier » ou « non premier ». Traitement : une boucle `Pour` (on connaît les bornes : de 2 à $\lfloor \sqrt{n} \rfloor$) contenant une conditionnelle qui teste le reste de la division grâce à l'opérateur **mod** (modulo).

```

Algorithme TestPrimalite
Variable n, d : Entier
Variable estPremier : Booleen
Debut
    Ecrire("Entrez un entier n >= 2 : ")

```

```

Lire(n)
estPremier <- Vrai      // on suppose n premier au depart
Si n < 2 Alors
    estPremier <- Faux
Sinon
    Pour d Allant de 2 a Entier(Racine(n)) Faire
        Si n mod d = 0 Alors
            estPremier <- Faux // un diviseur a ete trouve
        FinSi
    FinPour
FinSi
Si estPremier = Vrai Alors
    Ecrire(n, " est premier")
Sinon
    Ecrire(n, " n'est pas premier")
FinSi
Fin

```

La variable `estPremier` joue ici le rôle de **drapeau** (ou flag) : un booléen initialisé à `Vrai` que la boucle « fait tomber » dès qu'un diviseur est découvert.

EXEMPLE Test de primalité de 29, exécuté pas à pas

On applique l'algorithme avec $n = 29$. Comme $\sqrt{29} \approx 5,38$, la boucle teste les diviseurs d de 2 à 5 :

d	$29 \bmod d$	$= 0 ?$	Conséquence
2	1	Non	rien, on continue
3	2	Non	rien, on continue
4	1	Non	rien, on continue
5	4	Non	rien, on continue

Aucun diviseur entre 2 et 5 : le drapeau `estPremier` reste à `Vrai`. Conclusion : 29 est premier. En revanche, pour $n = 27$, le test $d = 3$ donne $27 \bmod 3 = 0$: le drapeau tombe et l'algorithme affiche « 27 n'est pas premier ».

2.3 Problème 2 : le PGCD par soustractions successives

Analyse. Le **PGCD** (plus grand commun diviseur) de deux entiers a et b repose sur une propriété remarquable : si $a > b$, alors $\text{pgcd}(a, b) = \text{pgcd}(a - b, b)$. En effet, tout diviseur commun à a et b divise aussi leur différence. On peut donc remplacer le plus grand des deux nombres par la différence, et recommencer, jusqu'à ce que les deux nombres soient égaux : cette valeur commune est le PGCD.

Choix de la structure. Ici, on ne connaît pas à l'avance le nombre de soustractions à effectuer : la boucle `Pour` est inadaptée. On utilise un `TantQue` dont la condition est « les deux nombres sont différents ».

Algorithme PGCD_Soustractions

Variable a, b : Entier

Debut

Ecrire("Entrez deux entiers strictement positifs : ")

Lire(a, b)

TantQue a <> b Faire

Si a > b Alors

a ← a - b // on retranche le plus petit du plus grand

Sinon

b ← b - a

FinSi

FinTantQue

Ecrire("Le PGCD est : ", a) // a et b sont égaux ici

Fin

EXEMPLE Exécution pas à pas : pgcd(48, 18) = 6

Traçons l'algorithme avec $a = 48$ et $b = 18$:

Étape	a	b	Action ($a \neq b$?)
Départ	48	18	$a > b : a \leftarrow 48 - 18 = 30$
1	30	18	$a > b : a \leftarrow 30 - 18 = 12$
2	12	18	$a < b : b \leftarrow 18 - 12 = 6$
3	12	6	$a > b : a \leftarrow 12 - 6 = 6$
4	6	6	$a = b$: fin de la boucle

La boucle s'arrête car $a = b = 6$. L'algorithme affiche : le PGCD est 6. Vérification : $48 = 6 \times 8$ et $18 = 6 \times 3$, et 8 et 3 sont premiers entre eux.

LESAVAIS-TU ? L'algorithme d'Euclide, un ancêtre de 2300 ans

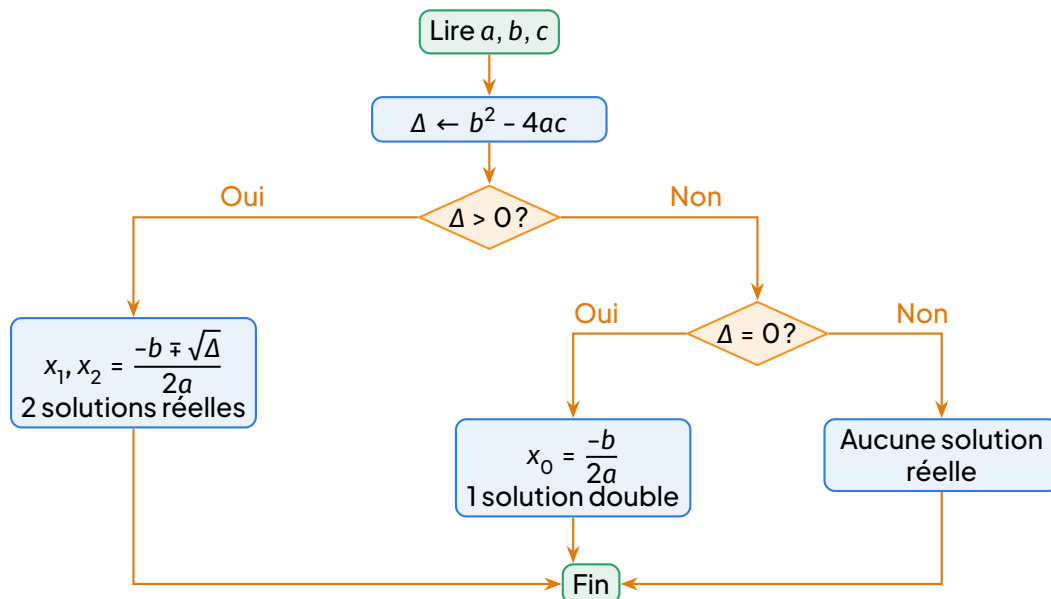
Le calcul du PGCD est décrit dans les *Éléments* du mathématicien grec Euclide, vers 300 av. J.-C. : c'est l'un des plus anciens algorithmes connus de l'humanité, et il est encore utilisé aujourd'hui ! La version moderne remplace les soustractions successives par des divisions euclidiennes (avec l'opérateur `mod`), ce qui la rend beaucoup plus rapide : $\text{pgcd}(48, 18)$ se calcule en deux divisions au lieu de quatre soustractions. Cet algorithme est au cœur de la cryptographie moderne, celle-là même qui sécurise tes transactions Mobile Money au Cameroun.

2.4 Problème 3 : résolution de l'équation du second degré

Analyse. Résoudre dans $\mathbb{R}$ l'équation $ax^2 + bx + c = 0$ (avec $a \neq 0$) est le problème type des **conditionnelles imbriquées**. On calcule le discriminant $\Delta = b^2 - 4ac$, puis on distingue trois cas

selon son signe :

- si $\Delta > 0$: deux solutions réelles distinctes $x_1 = \frac{-b - \sqrt{\Delta}}{2a}$ et $x_2 = \frac{-b + \sqrt{\Delta}}{2a}$;
- si $\Delta = 0$: une solution double $x_0 = \frac{-b}{2a}$;
- si $\Delta < 0$: aucune solution réelle.



Arbre de décision pour la résolution de $ax^2 + bx + c = 0$ selon le signe du discriminant Δ .

Algorithme SecondDegre

Variable a, b, c, delta, x1, x2, x0 : Reel

Debut

Ecrire("Entrez a, b, c (a non nul) : ")

Lire(a, b, c)

delta ← -b * b - 4 * a * c

Si delta > 0 Alors

x1 ← (-b - Racine(delta)) / (2 * a)

x2 ← (-b + Racine(delta)) / (2 * a)

Ecrire("Deux solutions : ", x1, " et ", x2)

Sinon

Si delta = 0 Alors

x0 ← -b / (2 * a)

Ecrire("Une solution double : ", x0)

Sinon

Ecrire("Pas de solution réelle")

FinSi

FinSi

Fin

Remarque bien l'**imbrication** : le second **Si** est placé à l'intérieur du **Si** du premier. Chaque **Si** possède son propre **FinSi** : c'est une exigence de syntaxe absolue.

EXERCICE RÉSOLU 2 Équation du second degré

Énoncé : écrire la trace de l'algorithme `SecondDegré` pour l'équation $2x^2 - 5x + 3 = 0$, puis pour $x^2 + x + 1 = 0$.

SOLUTION

Cas 1 : $a = 2, b = -5, c = 3$.

$$\Delta = b^2 - 4ac = (-5)^2 - 4 \times 2 \times 3 = 25 - 24 = 1 > 0$$

$$x_1 = \frac{-(-5) - \sqrt{1}}{2 \times 2} = \frac{5 - 1}{4} = 1, \quad x_2 = \frac{5 + 1}{4} = 1,5$$

L'algorithme affiche : « Deux solutions : 1 et 1,5 ». Vérification : $2(1)^2 - 5(1) + 3 = 0$ et $2(1,5)^2 - 5(1,5) + 3 = 4,5 - 7,5 + 3 = 0$. ✓

Cas 2 : $a = 1, b = 1, c = 1$: $\Delta = 1 - 4 = -3 < 0$. L'algorithme emprunte la branche « Non » puis « Non » de l'arbre de décision et affiche : « Pas de solution réelle ».

2.5 Problème 4 (physique) : le temps de chute libre

Analyse. Un objet lâché sans vitesse initiale d'une hauteur h (en mètres) touche le sol au bout d'un temps

$$t = \sqrt{\frac{2h}{g}}$$

où $g \approx 9,81 \text{ m/s}^2$ est l'accélération de la pesanteur. C'est un calcul purement **séquentiel** : lecture de h , calcul, affichage. Mais la physique nous offre une occasion en or de vérifier ce résultat par une boucle : la position de l'objet à l'instant t est $y(t) = h - \frac{1}{2}gt^2$. On peut simuler la chute pas à pas, par petits sauts de temps Δt , jusqu'à ce que $y \leq 0$.

```

Algorithme ChuteLibre
Constante g = 9,81 : Reel
Variable h, t, y, dt : Reel
Debut
    Ecrire("Hauteur de lacher (m) : ")
    Lire(h)
    // 1) Calcul direct (structure séquentielle)
    t <- Racine(2 * h / g)
    Ecrire("Temps theorique : ", t, " s")
    // 2) Verification par simulation (structure iterative)
    dt <- 0,01          // pas de temps : 10 ms
    t <- 0
    y <- h
    TantQue y > 0 Faire
        t <- t + dt

```

```

    y <- h - 0,5 * g * t * t // altitude a l'instant t
FinTantQue
Ecrire("Temps simule : ", t, " s")
Fin

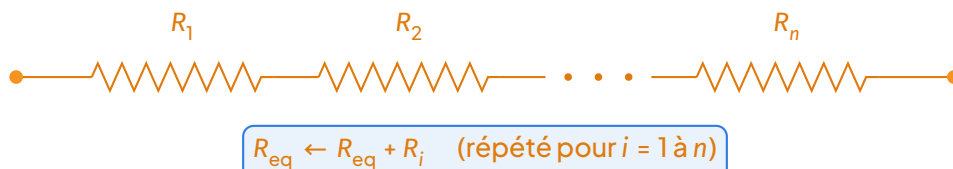
```

Pour $h = 100$ m, le calcul direct donne $t = \sqrt{200/9,81} \approx 4,52$ s, et la simulation par pas de 0,01 s confirme cette valeur à un centième près. Ici, le `TantQue` est indispensable : on ne sait pas à l'avance combien de pas de temps seront nécessaires.

2.6 Problème 5 (physique) : résistances en série et notion d'accumulateur

Analyse. En électricité, lorsque n résistances $R_1, R_2, \dots, R_n$ sont branchées **en série**, la résistance équivalente est simplement leur somme :

$$R_{eq} = R_1 + R_2 + \dots + R_n = \sum_{i=1}^n R_i$$



Des résistances en série : le même courant les traverse toutes, et la résistance équivalente s'obtient par accumulation des valeurs.

Comment traduire cette somme en algorithme ? On utilise un **accumulateur** : une variable, appelons-la R_{eq} , qui accumule les valeurs au fur et à mesure. Le schéma est toujours le même : **initialiser** l'accumulateur à 0 (élément neutre de l'addition) avant la boucle, puis, à chaque tour, lui **ajouter** la valeur courante.

```

Algorithme ResistanceSerie
Variable n, i : Entier
Variable R, Req : Reel
Debut
  Ecrire("Nombre de resistances : ")
  Lire(n)
  Req <- 0 // initialisation de l'accumulateur
  Pour i Allant de 1 a n Faire
    Ecrire("Valeur de R", i, " (ohms) : ")
    Lire(R)
    Req <- Req + R // accumulation
  FinPour
  Ecrire("Resistance equivalente : ", Req, " ohms")
Fin

```

Pour trois résistances de $10\ \Omega$, $22\ \Omega$ et $47\ \Omega$ (valeurs typiques d'un montage d'électronique), la trace donne : $R_{eq} = 0 \rightarrow 10 \rightarrow 32 \rightarrow 79\ \Omega$.

ATTENTION Erreur fréquente : l'accumulateur non initialisé

Oublier la ligne `Req <- 0` avant la boucle est l'une des erreurs les plus classiques — et les plus graves. Une variable déclarée mais non initialisée contient une valeur imprévisible : la somme calculée sera fausse, sans aucun message d'erreur pour te prévenir ! Retiens la règle d'or : tout accumulateur est initialisé avant la boucle, à 0 pour une somme, à 1 pour un produit. Autre piège : placer l'initialisation à l'intérieur de la boucle, ce qui remettrait le compteur à zéro à chaque tour et ne conserverait que la dernière valeur saisie.

À RETENIR Ce qu'il faut retenir de cette section

- La résolution d'un problème suit quatre étapes : comprendre, découper (entrées / traitement / sorties), coder, tracer.
- Le choix de la structure dépend du problème : `Pour` si le nombre de tours est connu (primauté, somme de n valeurs), `TantQue` s'il dépend d'une condition (PGCD, simulation de chute), conditionnelles imbriquées pour les cas multiples (second degré).
- L'opérateur `mod` (reste de la division) est l'outil central des problèmes de divisibilité.
- Un accumulateur s'initialise toujours avant la boucle : à 0 pour une somme, à 1 pour un produit.

EXERCICE 2 À toi de jouer

1. Écris un algorithme qui calcule la factorielle d'un entier n saisi ($n! = 1 \times 2 \times \dots \times n$). Quelle valeur d'initialisation choisis-tu pour l'accumulateur, et pourquoi ?
2. Un corps est lancé verticalement vers le haut avec une vitesse initiale v_0 . Sa hauteur à l'instant t est $y(t) = v_0 t - \frac{1}{2} g t^2$. Écris un algorithme qui détermine, par simulation avec un `TantQue`, l'instant où le corps retombe au sol ($y \leq 0$).
3. Modifie l'algorithme `ResistanceSerie` pour qu'il calcule la résistance équivalente de n résistances en parallèle, sachant que $\frac{1}{R_{eq}} = \sum_{i=1}^n \frac{1}{R_i}$.

CORRIGÉ Exercice 1 (factorielle)

L'accumulateur `F` doit être initialisé à 1 (élément neutre de la multiplication) : l'initialiser à 0 donnerait un résultat nul, car 0 multiplié par tout nombre vaut 0.

```
Algorithm Factorielle
Variable n, i, F : Entier
Debut
  Lire(n)
  F ← 1
  Pour i Allant de 2 à n Faire
    F ← F * i
  FinPour
  Ecrire(n, " ! = ", F)
Fin
```

Tu maîtrises maintenant l'art d'assembler les structures de contrôle pour résoudre des problèmes réels. Mais tous ces algorithmes manipulaient des variables isolées. Comment faire lorsqu'on doit traiter une collection de données — les notes d'une classe entière, les températures d'un mois ? C'est toute la question des **structures de données**, que nous aborderons dans la prochaine section.

3 LES STRUCTURES DE DONNÉES : PANORAMA

Dans la section précédente, nous avons écrit des algorithmes qui résolvent des problèmes de mathématiques et de physique : équation du second degré, calcul d'une moyenne, conversion d'unités. Dans tous ces algorithmes, les données étaient simples : un nombre entier, un réel, un caractère. Une variable ne contenait qu'une **seule valeur** à la fois. Mais la vie réelle est plus riche : la bibliothèque du lycée de Bafoussam ne gère pas un livre, mais cinq cents ; la secrétaire du proviseur ne manipule pas une moyenne, mais le dossier complet de chaque élève (nom, âge, classe, notes) ; le logiciel de traitement de texte doit se souvenir de toutes tes dernières actions pour pouvoir les annuler. Comment organiser toutes ces données en mémoire de façon cohérente et efficace ? C'est exactement l'objet des **structures de données**.

3.1 Qu'est-ce qu'une structure de données ?

DÉFINITION Structure de données

Une **structure de données** est un mode d'organisation des données en mémoire qui permet de les stocker, de les regrouper et de les manipuler efficacement (recherche, insertion, suppression, parcours). Elle définit à la fois la façon dont les données sont rangées et les opérations que l'on peut effectuer sur elles.

L'idée fondamentale est la suivante : il n'existe pas de structure « meilleure » dans l'absolu. Chaque structure est adaptée à un **type de problème**. Ranger des livres dans une armoire, sur une pile posée au sol, ou en file dans un couloir ne donne pas les mêmes possibilités : dans l'armoire tu accèdes directement au livre que tu veux ; sur la pile tu ne peux prendre que celui du

dessus ; dans la file tu peux facilement insérer un nouveau livre entre deux autres. De même, en algorithmique, le choix de la structure conditionne la simplicité et l'efficacité de l'algorithme.

Au programme de Première, nous retenons quatre structures classiques : le **tableau**, l'**enregistrement**, la **pile** et la **liste**. Étudions-les une à une.

3.2 Le tableau : des cases de même type, accessibles par indice

DÉFINITION Tableau

Un **tableau** est une structure de données qui regroupe, sous un même nom, une collection d'éléments de même type, rangés dans des cases mémoire contiguës et numérotées. Chaque élément est accessible directement grâce à son **indice** (sa position dans le tableau).

Imagine les cases numérotées d'un casier scolaire : tous les casiers sont identiques (même type de contenu), et pour ouvrir celui qui t'intéresse, il suffit de connaître son numéro. C'est exactement le fonctionnement d'un tableau : si `Notes` est un tableau de 5 réels, alors `Notes[3]` désigne directement l'élément d'indice 3, sans avoir à parcourir les précédents.

PROPRIÉTÉ Caractéristiques du tableau

- Tous les éléments sont de même type (tous entiers, tous réels, toutes chaînes de caractères).
- L'accès à un élément est direct grâce à l'indice : on parle d'accès indexé.
- La taille est fixée à la déclaration : un tableau déclaré pour 20 élèves ne pourra pas en contenir 21.

3.3 L'enregistrement : des champs de types différents

Un tableau impose que tous les éléments soient de même type. Or, pour décrire un élève du lycée, il faut mélanger des informations de natures différentes : son nom (chaîne de caractères), son âge (entier), sa moyenne (réel). Impossible de tout mettre dans un seul tableau ! La solution est l'**enregistrement**.

DÉFINITION Enregistrement

Un **enregistrement** est une structure de données qui regroupe sous un même nom plusieurs **champs**, pouvant être de types différents. Chaque champ possède un nom qui permet d'y accéder individuellement.

EXEMPLE L'enregistrement Élève

Voici comment on déclare et utilise, en pseudo-code, un enregistrement décrivant un élève de Première C :

```
Type Eleve = Enregistrement
|   nom   : Chaîne
|   age    : Entier
|   moyenne : Reel
| FinEnregistrement

Variable e : Eleve

Debut
|   e.nom   <- "Ngo Bell"
|   e.age   <- 16
|   e.moyenne <- 13,75
|   Ecrire(e.nom, " a pour moyenne ", e.moyenne)
| Fin
```

On accède à chaque champ par la notation pointée : `e.nom`, `e.age`, `e.moyenne`. L'affichage produit : Ngo Bell a pour moyenne 13,75.

ATTENTION Ne pas confondre tableau et enregistrement

C'est une confusion très fréquente à l'examen :

- Le tableau regroupe des éléments de même type, accessibles par indice : `Notes[4]`.
- L'enregistrement regroupe des champs de types différents, accessibles par nom de champ : `e.moyenne`.

Les deux se combinent souvent : pour gérer toute une classe, on utilise un tableau d'enregistrements, par exemple `Classe[12].moyenne` désigne la moyenne de l'élève d'indice 12.

3.4 La pile : dernier entré, premier sorti (LIFO)

Dans la cantine du lycée, les assiettes propres sont empilées les unes sur les autres. Quand un élève se sert, il prend l'**assiette du dessus** : celle qui a été posée en dernier. Quand on lave une nouvelle assiette, on la pose au sommet. On ne peut jamais prendre directement une assiette du milieu sans faire tomber les autres ! Cette organisation si particulière est une structure de données fondamentale : la **pile**.

DÉFINITION Pile

Une **pile** est une structure de données dans laquelle on ne peut ajouter ou retirer un élément qu'à une seule extrémité, appelée le **sommet**. Elle obéit au principe **LIFO** (Last In, First Out) : le dernier élément entré est le premier à sortir. Les deux opérations de base sont :

- **empiler** (push) : ajouter un élément au sommet ;
- **dépiler** (pop) : retirer l'élément situé au sommet.

EXEMPLE Déroutement d'opérations sur une pile

Partons d'une pile vide et effectuons : empiler 5, empiler 8, empiler 3, dépiler, dépiler.

Opération	État de la pile (sommet à droite)	Valeur dépilée
empiler(5)	5	—
empiler(8)	5, 8	—
empiler(3)	5, 8, 3	—
dépiler()	5, 8	3
dépiler()	5	8

On vérifie le principe LIFO : le 3, entré en dernier, est bien sorti en premier.

LESAVAIS-TU ? La pile et la touche « Annuler » (Ctrl+Z)

La pile est au cœur du fonctionnement de la commande Annuler (Ctrl+Z) de tes logiciels préférés (traitement de texte, tableur, éditeur d'images). Chaque action que tu effectues (taper un mot, supprimer un paragraphe, changer une couleur) est empilée. Quand tu presses Ctrl+Z, le logiciel dépile la dernière action et l'annule : c'est forcément la plus récente, conformément au principe LIFO. Appuie plusieurs fois : les actions sont annulées dans l'ordre inverse de leur exécution. La pile intervient aussi dans le bouton « Précédent » du navigateur web et dans l'exécution des appels de fonctions par le processeur !

3.5 La liste : des éléments chaînés

Revenons à notre bibliothèque scolaire. Avec un tableau de 500 livres, insérer un nouveau livre « au milieu » (par exemple pour garder l'ordre alphabétique) oblige à décaler tous les livres suivants d'une case : c'est long et coûteux. La **liste** résout ce problème en reliant les éléments les uns aux autres, comme les maillons d'une chaîne.

DÉFINITION Liste chaînée

Une **liste chaînée** est une structure de données formée d'une suite d'éléments appelés **maillons** (ou nœuds). Chaque maillon contient :

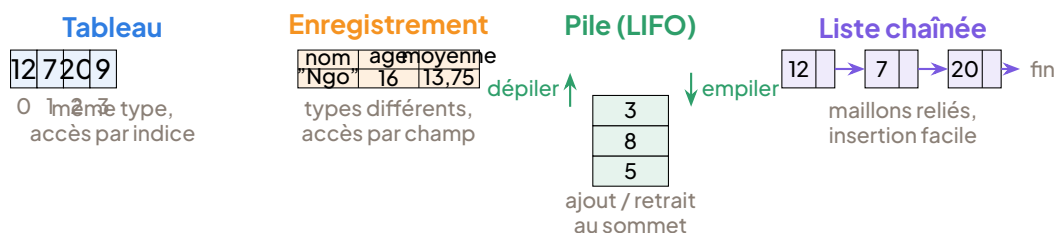
- une donnée (la valeur stockée) ;
- un **lien** (ou pointeur) vers le maillon suivant.

On accède à la liste par son **premier maillon** (la tête), puis on la parcourt en suivant les liens de proche en proche.

PROPRIÉTÉ Avantages et limites de la liste chaînée

- Insertion et suppression facilitées : pour insérer un maillon entre deux autres, il suffit de « rediriger » deux liens, sans déplacer les autres éléments.
- Taille dynamique : la liste grandit ou rétrécit au fur et à mesure des besoins, contrairement au tableau dont la taille est fixe.
- Limite : pas d'accès direct par indice ! Pour atteindre le 10^e élément, il faut partir de la tête et suivre 9 liens : on parle d'accès séquentiel.

3.6 Schéma comparatif des quatre structures



Les quatre structures de données au programme : tableau (accès indexé), enregistrement (champs nommés), pile (LIFO) et liste chaînée (maillons reliés par des liens).

3.7 Comment choisir la bonne structure ?

Le choix d'une structure de données n'est jamais anodin : c'est souvent lui qui rend un algorithme simple ou compliqué, rapide ou lent. Voici une démarche à suivre.

MÉTHODE Choisir une structure de données adaptée à un problème

1. Identifier les données : sont-elles toutes de même type, ou de types différents ?
2. Identifier les opérations principales : accès direct à un élément ? parcours complet ? insertions et suppressions fréquentes ? traitement dans l'ordre inverse d'arrivée ?
3. Appliquer les règles de correspondance :
 - même type + accès direct par position $\Rightarrow$ tableau ;
 - types différents décrivant un même objet $\Rightarrow$ enregistrement ;
 - traitement « dernier arrivé, premier servi » $\Rightarrow$ pile ;
 - insertions/suppressions fréquentes, taille variable $\Rightarrow$ liste.
4. Vérifier la cohérence : combiner si nécessaire (tableau d'enregistrements pour une classe entière).

EXERCICE RÉSOLU 3**Quelle structure pour quel problème ?**

Pour chacune des situations suivantes, indique la structure de données la plus adaptée et justifie ton choix :

1. Mémoriser les températures relevées chaque jour du mois de mars à Yaoundé (31 valeurs réelles) pour calculer ensuite la température maximale.
2. Gérer le dossier scolaire d'un candidat au Probatoire : nom, date de naissance, série, notes des épreuves.
3. Mémoriser les pages web visitées pour permettre le retour à la page précédente.
4. Gérer les adhérents d'un club informatique dont le nombre varie constamment, avec des inscriptions et des désinscriptions en cours d'année.

SOLUTION

Solution détaillée.

1. Tableau de 31 réels. Les données sont toutes de même type (des réels), en nombre connu à l'avance, et l'accès direct par indice (`Temp[15]` pour le 15 mars) est très pratique. Le parcours pour trouver le maximum se fait avec une simple boucle `Pour`.
2. Enregistrement. Les informations sont de types différents (chaînes, date, réels) mais décrivent un seul et même objet : le candidat. On accède à chaque information par son nom de champ (`candidat.serie`).
3. Pile. Chaque page visitée est empilée ; le bouton « Précédent » dépile la dernière page, c'est-à-dire la plus récente : c'est exactement le principe LIFO.
4. Liste chaînée. Le nombre d'adhérents est inconnu et variable ; les inscriptions (insertions) et désinscriptions (suppressions) sont fréquentes, et la liste les réalise sans dé-

placer les autres éléments.

À RETENIR L'essentiel de la section

- Une **structure de données** organise les données en mémoire pour les manipuler efficacement.
- Le **tableau** : éléments de même type, accès direct par indice, taille fixe.
- L'**enregistrement** : champs de types différents, accès par nom de champ.
- La **pile** : principe LIFO, opérations empiler et dépiler au sommet.
- La **liste chaînée** : maillons reliés par des liens, insertions et suppressions faciles, mais accès séquentiel.
- Le choix de la structure dépend du problème à résoudre et des opérations dominantes.

EXERCICE 3 À toi de jouer

1. Donne deux situations de la vie courante (hors informatique) qui fonctionnent comme une pile, et deux qui fonctionnent comme une liste.
2. On veut gérer les notes de toutes les matières de tous les élèves d'une classe de Première D. Propose une organisation des données en combinant plusieurs structures vues dans cette section.
3. On empile successivement les valeurs 4, 9, 1, 7 dans une pile vide, puis on dépile deux fois, puis on empile 5. Quel est le contenu final de la pile ? Quelle valeur se trouve au sommet ?

Dans la section suivante, nous approfondirons la première de ces structures, la plus utilisée au programme de Première : le **tableau à une dimension**, avec sa déclaration, son parcours et les algorithmes classiques de recherche.

4

LE TYPE TABLEAU À UNE DIMENSION

Dans la section précédente, nous avons dressé un panorama des structures de données : enregistrements, tableaux, piles, listes. Nous avons vu qu'une structure de données permet de regrouper et d'organiser plusieurs informations sous un même nom. Parmi toutes ces structures, une seule est étudiée en profondeur en Première : le **tableau à une dimension**. C'est la structure la plus simple, la plus utilisée, et celle qui ouvre la porte à tous les algorithmes de trai-

tement de données en série : notes d'une classe, températures du mois, prix des articles d'une boutique, effectifs des villages d'un arrondissement.

4.1 Pourquoi un tableau ? Le problème des variables multiples

Imaginons que le proviseur d'un lycée de Bafoussam te demande d'écrire un algorithme qui calcule la moyenne de classe en informatique. La classe compte 50 élèves. Avec ce que tu sais déjà faire, tu déclarerais 50 variables : `note1`, `note2`, ..., `note50`. Deux catastrophes apparaissent immédiatement :

- il faut écrire 50 déclarations et 50 instructions de lecture : l'algorithme devient interminable ;
- impossible d'utiliser une boucle `Pour`, car les noms des variables ne se « calculent » pas : on ne peut pas dire « lis la variable numéro *i* ».

Il faut donc un mécanisme qui permette de désigner chaque donnée par un nom commun et un numéro. C'est exactement ce que fait le tableau : `T[1]`, `T[2]`, ..., `T[50]` sont 50 variables qui portent le même nom `T` et qui ne diffèrent que par leur numéro, appelé **indice**. Comme l'indice est une valeur entière, il peut être contenu dans une variable *i* : on peut alors parcourir tout le tableau avec une simple boucle.

DÉFINITION Tableau à une dimension et indice

Un **tableau à une dimension** (ou vecteur) est une structure de données qui regroupe, sous un même nom, un nombre fixe d'éléments de même type, rangés dans des cases consécutives et repérés par un numéro appelé **indice**. Chaque élément du tableau s'utilise exactement comme une variable ordinaire : on peut le lire, l'écrire, l'afficher, le comparer.

PROPRIÉTÉ Syntaxe de déclaration d'un tableau

En algorithmique, un tableau se déclare dans la partie déclarative en précisant trois choses : son nom, l'intervalle des indices (premier et dernier indice) et le type des éléments :

`nom : tableau[premierIndice..dernierIndice] de type`

Exemple : `T : tableau[1..10] de reels` déclare un tableau `T` de 10 cases numérotées de 1 à 10, chacune pouvant contenir un nombre réel. Le nombre d'éléments, appelé **taille** du tableau, vaut : `dernierIndice - premierIndice + 1`.

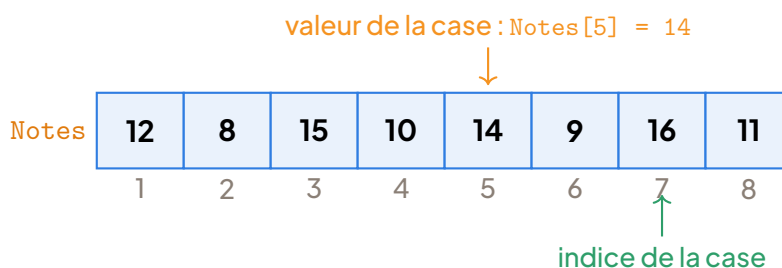
ATTENTION Les bornes sont intangibles

L'intervalle des indices est fixé une fois pour toutes à la déclaration. Si `T` est déclaré

tableau[1..10] de reals, alors les seules cases légales sont T[1] à T[10]. Écrire T[0] ou T[11] est une erreur appelée **dépassement de capacité** (ou dépassement des bornes) : en langage C, elle ne provoque souvent aucun message d'erreur mais corrompt silencieusement d'autres données en mémoire. Vérifie toujours que tes indices restent entre le premier et le dernier indice déclarés.

4.2 Représentation en mémoire : cases, indices, valeurs

Un tableau se représente comme une rangée de cases. Chaque case possède un **indice** (son adresse dans le tableau) et contient une **valeur**. Voici le tableau `Notes` des notes d'informatique de 8 élèves d'une classe de Première D :



Le tableau `Notes` : tableau[1..8] de reals en mémoire : 8 cases numérotées de 1 à 8, chacune contenant une note.

L'accès à un élément se note `Notes[i]` : c'est la valeur de la case d'indice i . Ainsi `Notes[3]` vaut 15, et l'affectation `Notes[2] ← 13` remplace le 8 par un 13. L'indice peut être une constante (`Notes[3]`), une variable (`Notes[i]`) ou même une expression (`Notes[i+1]`), à condition que sa valeur reste entre 1 et 8.

4.3 Remplir un tableau par lecture au clavier

Pour saisir les valeurs d'un tableau, on utilise une boucle `Pour` qui fait varier l'indice de la première à la dernière case. À chaque tour, on lit une valeur et on la range dans la case courante.

```

Algorithme Remplissage
Variable
    Notes : tableau[1..8] de reals
    i : entier
Debut
    Pour i allant de 1 à 8 Faire
        Ecrire("Entrez la note de l'eleve", i, " : ")
        Lire(Notes[i]) // la valeur lue est rangee dans la case i
    FinPour
Fin
  
```

Au premier tour, i vaut 1 et la note saisie est rangée dans `Notes[1]` ; au deuxième tour, dans `Notes[2]`,

et ainsi de suite jusqu'à `Notes[8]`. C'est toute la puissance du tableau : **une seule instruction** `Lire(Notes[i])` remplit les 8 cases.

4.4 Parcourir un tableau : la démarche générale

Remplir, afficher, sommer, rechercher : tous ces traitements reposent sur le même schéma, le **parcours séquentiel** du tableau.

MÉTHODE Parcourir un tableau en trois étapes

1. Initialiser ce qui est nécessaire avant la boucle : un compteur, une somme à 0, une variable résultat, etc.
2. Boucler avec un indice i allant du premier au dernier indice (Pour i allant de 1 à n).
3. Traiter la case courante `T[i]` dans le corps de la boucle (l'afficher, l'ajouter à une somme, la comparer à un candidat maximum...), puis exploiter le résultat après la boucle si besoin.

L'affichage simple des éléments en est l'illustration la plus directe :

```
Pour i allant de 1 à 8 Faire
    Ecrire("Notes[" , i, " ] = ", Notes[i])
FinPour
```

4.5 Somme et moyenne des éléments

Pour calculer la somme des éléments, on applique la méthode : on initialise un accumulateur S à 0, puis à chaque tour de boucle on ajoute la case courante : $S \leftarrow S + \text{Notes}[i]$. La moyenne s'obtient ensuite en divisant la somme par le nombre d'éléments.

```
Algorithme MoyenneClasse
Variable
    Notes : tableau[1..8] de reels
    i : entier
    S, Moy : reels
Debut
    // 1. Remplissage
    Pour i allant de 1 à 8 Faire
        Ecrire("Note de l'eleve ", i, " : ")
        Lire(Notes[i])
    FinPour
    // 2. Calcul de la somme
    S ← 0
    Pour i allant de 1 à 8 Faire
        S ← S + Notes[i]
    FinPour
    // 3. Moyenne et affichage
    Moy ← S / 8
```

```
Ecrire("La moyenne de la classe est : ", Moy)
```

```
Fin
```

EXEMPLE Calcul chiffré de la moyenne

Prenons les six notes {12, 8, 15, 10, 14, 9}. L'accumulateur évolue ainsi :

$$S = 0 \rightarrow 12 \rightarrow 20 \rightarrow 35 \rightarrow 45 \rightarrow 59 \rightarrow 68.$$

La moyenne vaut alors :

$$\text{Moy} = \frac{S}{n} = \frac{68}{6} \approx 11,33.$$

L'algorithme affiche donc : « La moyenne de la classe est : 11,33 ».

ATTENTION Initialiser l'accumulateur

L'erreur classique est d'oublier $S \leftarrow 0$ avant la boucle. La variable S contient alors une valeur imprévisible et le résultat est faux. Toute variable qui « accumule » (somme, produit, compteur) doit être initialisée avant la boucle, jamais dedans.

4.6 Recherche du maximum et du minimum

Chercher la plus grande note d'une classe est un problème très naturel. La stratégie est celle du « champion provisoire » : on déclare champion le premier élément, puis on parcourt le reste du tableau ; chaque fois qu'une case dépasse le champion, elle lui prend sa place.

```
Algorithme Maximum
```

```
Variable
```

```
Notes : tableau[1..8] de reels
```

```
i : entier
```

```
Max : reel
```

```
Debut
```

```
// (le tableau est suppose deja rempli)
```

```
Max ← Notes[1] // le premier element est champion provisoire
```

```
Pour i allant de 2 a 8 Faire
```

```
Si Notes[i] > Max Alors
```

```
Max ← Notes[i] // nouveau champion
```

```
FinSi
```

```
FinPour
```

```
Ecrire("La meilleure note est : ", Max)
```

```
Fin
```

Pour le **minimum**, c'est identique : on initialise $\text{Min} \leftarrow \text{Notes}[1]$ et on remplace le test par $\text{Notes}[i] < \text{Min}$.

ATTENTION Ne pas initialiser le maximum à 0

Beaucoup d'élèves écrivent $\text{Max} \leftarrow 0$. C'est faux : si toutes les valeurs étaient négatives (par exemple des températures hivernales), l'algorithme répondrait 0, qui n'est même pas dans le tableau ! La seule initialisation correcte est $\text{Max} \leftarrow T[1]$, car le maximum d'un tableau appartient forcément au tableau.

Voici la trace complète de la recherche du maximum sur six valeurs :

Tour i	Notes[i]	Test Notes[i] > Max ?	Max après le tour
-	-	initialisation : $\text{Max} \leftarrow \text{Notes}[1]$	12
2	8	$8 > 12$? non	12
3	15	$15 > 12$? oui, $\text{Max} \leftarrow 15$	15
4	10	$10 > 15$? non	15
5	14	$14 > 15$? non	15
6	9	$9 > 15$? non	15

Trace de la recherche du maximum sur {12, 8, 15, 10, 14, 9} : le maximum trouvé est 15.

4.7 Recherche séquentielle d'une valeur

Autre problème fondamental : le chef d'une boutique veut savoir si le code article 457 figure dans son stock, et à quelle position. On ne sait rien de l'ordre des valeurs, donc la seule méthode possible est la **recherche séquentielle** : on examine les cases une par une, de la première à la dernière, jusqu'à trouver la valeur cherchée ou épuiser le tableau.

Algorithme RechercheSequentielle

Variable

T : tableau[1..8] de entiers

i , X : entiers

Trouve : booléen

Debut

// (le tableau est suppose deja rempli)

Ecrire("Valeur a chercher : ")

Lire(X)

Trouve $\leftarrow$ Faux

$i \leftarrow 1$

Tant que ($i \leq 8$) et (Trouve = Faux) Faire

Si $T[i] = X$ Alors

Trouve $\leftarrow$ Vrai

Sinon

$i \leftarrow i + 1$ // on passe a la case suivante

FinSi

FinTantQue

```
Si Trouve = Vrai Alors
    Ecrire(X, " trouve a la position ", i)
Sinon
    Ecrire(X, " absent du tableau")
FinSi
Fin
```

On utilise ici une boucle `Tant que` plutôt qu'un `Pour`, car on veut **s'arrêter dès que la valeur est trouvée**, sans examiner inutilement les cases restantes. Remarque la condition double : `i <= 8` garantit qu'on ne dépasse jamais la dernière borne, même si la valeur est absente.

EXERCICE RÉSOLU 4 Trace de la recherche séquentielle

Soit le tableau $T = [12, 8, 15, 10, 14, 9]$ ($n = 6$). On cherche $X = 14$. Donne la trace de l'algorithme puis le résultat affiché. Même question avec $X = 7$.

SOLUTION

Recherche de 14 :

- $i = 1 : T[1] = 12 \neq 14$, on avance;
- $i = 2 : T[2] = 8 \neq 14$, on avance;
- $i = 3 : T[3] = 15 \neq 14$, on avance;
- $i = 4 : T[4] = 10 \neq 14$, on avance;
- $i = 5 : T[5] = 14 = 14$: Trouve devient Vrai, la boucle s'arrête.

L'algorithme affiche : « 14 trouve a la position 5 » (5 comparaisons effectuées).

Recherche de 7 : les cases 1 à 6 sont examinées sans succès ; à $i = 7$, la condition `i <= 6` devient fausse et la boucle s'arrête avec `Trouve = Faux`. L'algorithme affiche : « 7 absent du tableau » (6 comparaisons, soit le maximum possible).

LESAVAIS-TU ? De la recherche séquentielle à la recherche dichotomique

Dans un tableau de n cases, la recherche séquentielle coûte au plus n comparaisons (quand la valeur est absente ou en dernière position). Pour un tableau de 1 000 000 d'articles, cela fait jusqu'à 1 000 000 de tests ! En Terminale, tu découvriras la **recherche dichotomique**, applicable aux tableaux triés : on compare la valeur cherchée à l'élément du milieu, ce qui élimine d'un coup la moitié du tableau, puis on recommence sur la moitié restante. Le nombre de comparaisons tombe à environ $\log_2(n)$: pour 1 000 000 d'éléments, seulement 20 comparaisons suffisent, car $2^{20} \approx 10^6$. C'est le principe utilisé quand tu cherches un mot dans un dictionnaire : tu ne lis pas toutes les pages !

4.8 Exécution manuelle complète : les notes d'une classe de 8 élèves

Terminons par un algorithme complet qui enchaîne tous les traitements étudiés : remplissage, affichage, somme, moyenne, maximum et minimum, pour les 8 notes {12, 8, 15, 10, 14, 9, 16, 11} de notre classe de Première D.

EXPÉRIENCE

Exécuter à la main un algorithme à tableaux

Objectif : vérifier, sans ordinateur, le comportement d'un algorithme complet sur tableaux.

Algorithme à exécuter :

```

Algorithme BilanClasse
Variable
  Notes : tableau[1..8] de reels
  i : entier
  S, Moy, Max, Min : reels
Debut
  Pour i allant de 1 a 8 Faire
    Ecrire("Note ", i, " : ")
    Lire(Notes[i])
  FinPour
  S ← 0
  Max ← Notes[1]
  Min ← Notes[1]
  Pour i allant de 1 a 8 Faire
    S ← S + Notes[i]
    Si Notes[i] > Max Alors
      Max ← Notes[i]
    FinSi
    Si Notes[i] < Min Alors
      Min ← Notes[i]
    FinSi
  FinPour
  Moy ← S / 8
  Ecrire("Somme = ", S, " Moyenne = ", Moy)
  Ecrire("Maximum = ", Max, " Minimum = ", Min)
Fin
  
```

Manipulation : remplis une table d'exécution avec une colonne par variable (i , $\text{Notes}[i]$, S , Max , Min) et une ligne par tour de boucle, en saisissant mentalement les 8 notes.

Observations attendues : la somme évolue ainsi : $12 \rightarrow 20 \rightarrow 35 \rightarrow 45 \rightarrow 59 \rightarrow 68 \rightarrow 84 \rightarrow 95$. Le maximum passe de 12 à 15 (tour 3) puis à 16 (tour 7) ; le minimum descend de 12 à 8 (tour 2) et n'en bouge plus.

Interprétation : l'algorithme affiche finalement :

$$S = 95, \quad \text{Moy} = \frac{95}{8} = 11,875, \quad \text{Max} = 16, \quad \text{Min} = 8.$$

Remarque qu'une seule boucle suffit pour cumuler la somme, le maximum et le minimum :

c'est plus efficace que trois parcours séparés.

À RETENIR L'essentiel sur les tableaux à une dimension

- Un tableau regroupe sous un même nom des éléments de même type, repérés par un indice ; on le déclare par `T : tableau[1..n]` de type.
- Tout accès `T[i]` doit respecter les bornes : $1 \leq i \leq n$; sinon, c'est le dépassement de capacité.
- Tous les traitements suivent le même schéma : initialiser, boucler de la première à la dernière case, traiter `T[i]`.
- Somme : accumulateur initialisé à 0 ; maximum/minimum : initialisés à `T[1]` ; recherche séquentielle : boucle `Tant que` avec arrêt dès que la valeur est trouvée, en au plus n comparaisons.

EXERCICE 4 À toi de jouer

Écris un algorithme qui remplit un tableau `Temp` des 12 températures moyennes mensuelles de la ville de Yaoundé, puis qui calcule et affiche : (a) la température moyenne annuelle ; (b) le numéro du mois le plus chaud (on supposera les températures toutes distinctes).

CORRIGÉ Exercice 1

```

Algorithme TemperaturesYaounde
Variable
    Temp : tableau[1..12] de reels
    i, MoisChaud : entiers
    S, Moy, Max : reels
Debut
    Pour i allant de 1 à 12 Faire
        Ecrire("Temperature du mois ", i, " : ")
        Lire(Temp[i])
    FinPour
    S <- 0
    Max <- Temp[1]
    MoisChaud <- 1
    Pour i allant de 1 à 12 Faire
        S <- S + Temp[i]
        Si Temp[i] > Max Alors

```

```
Max <- Temp[i]
MoisChaud <- i // on memorise la position du maximum
FinSi
FinPour
Moy <- S / 12
Ecrire("Moyenne annuelle : ", Moy)
Ecrire("Le mois le plus chaud est le mois ", MoisChaud)
Fin
```

L'astuce consiste à mémoriser, en plus du maximum, l'indice de la case où il a été rencontré : c'est une technique très courante quand on veut la position et pas seulement la valeur.

5 FONCTIONS ET PROCÉDURES

Dans la section précédente, nous avons découvert le **type tableau**, qui permet de regrouper plusieurs valeurs sous un même nom. C'était déjà une première façon d'organiser les données. Mais l'organisation d'un programme ne concerne pas seulement les données : elle concerne aussi les traitements. Imagine que, dans l'algorithme de facturation d'un cybercafé à Douala, le calcul du prix à payer apparaisse à dix endroits différents. Si le gérant modifie son tarif, il faudra corriger dix fois le même calcul, avec le risque d'en oublier un. Il existe une solution élégante : écrire ce calcul une seule fois, lui donner un nom, puis l'appeler chaque fois qu'on en a besoin. C'est exactement le rôle des **fonctions** et des **procédures**.

5.1 Pourquoi découper un programme ?

EXEMPLE Le problème du programme qui se répète

Un élève écrit un algorithme qui calcule le carré de trois nombres saisis. Il écrit trois fois le même calcul `resultat ← n * n`. Le lendemain, le professeur demande le cube au lieu du carré : il faut modifier trois lignes, sans en oublier une seule. Avec une fonction `Carre(x)` écrite une seule fois, une seule modification suffit.

Découper un programme en sous-programmes (fonctions et procédures) apporte trois bénéfices majeurs :

- **Réutilisation** : un traitement écrit une fois peut être appelé autant de fois que nécessaire, avec des valeurs différentes à chaque appel.
- **Lisibilité** : l'algorithme principal devient court et clair ; chaque sous-programme porte un nom qui décrit son rôle (`CalculerMoyenne`, `AfficherBulletin`...).
- **Correction facilitée** : si une erreur existe, on sait exactement dans quel sous-programme elle se trouve ; on teste chaque morceau séparément avant de les assembler.

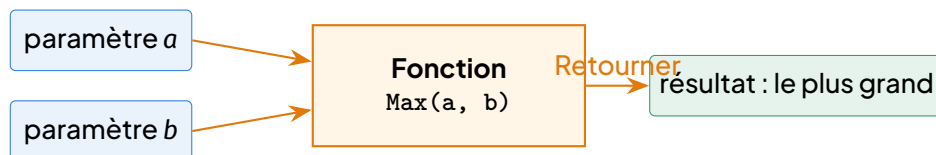
À RETENIR Principe du découpage

Un programme complexe se construit comme une équipe : l'algorithme principal donne les ordres, et chaque sous-programme exécute une tâche précise. On distingue deux sortes de sous-programmes : la **fonction**, qui calcule et retourne un résultat, et la **procédure**, qui effectue une action sans retourner de valeur.

5.2 La fonction : un sous-programme qui retourne un résultat**DÉFINITION** Fonction

Une **fonction** est un sous-programme qui reçoit zéro, une ou plusieurs valeurs en entrée (les **paramètres**), effectue un traitement, et retourne un unique résultat d'un type donné, grâce à l'instruction `Retourner`.

On peut voir une fonction comme une boîte noire : on y introduit des valeurs, elle effectue son travail à l'intérieur, et elle nous rend un résultat. L'utilisateur de la fonction n'a pas besoin de connaître le détail du calcul : il lui suffit de savoir ce qu'elle attend et ce qu'elle rend.



Vision « boîte noire » d'une fonction : les paramètres entrent, le traitement s'effectue, un unique résultat sort.

MÉTHODE Déclarer une fonction en cinq étapes

1. Choisir un nom explicite qui décrit ce que la fonction calcule (ex. Carre, Max, Facture).
2. Déclarer les paramètres avec leur type : ce sont les valeurs dont la fonction a besoin pour travailler.
3. Préciser le type du résultat retourné (entier, réel, booléen, chaîne...).
4. Écrire le corps : les instructions du calcul, en utilisant les paramètres comme des variables.
5. Terminer par l'instruction `Retourner` suivie de l'expression du résultat.

Syntaxe générale en pseudo-code :

```
Fonction NomFonction(param1 : Type1 ; param2 : Type2) : TypeResultat
Variable
    // variables locales si nécessaire
Debut
    // instructions du calcul
    Retourner expression
Fin
```

EXEMPLE Deux fonctions classiques

```
Fonction Carre(x : Reel) : Reel
Debut
    Retourner x * x
Fin

Fonction Max(a : Reel ; b : Reel) : Reel
Debut
    Si a >= b Alors
        Retourner a
    Sinon
        Retourner b
    FinSi
Fin
```

La fonction `Carre` reçoit un réel et retourne son carré. La fonction `Max` reçoit deux réels et retourne le plus grand des deux.

ATTENTION L'erreur n° 1 : oublier `Retourner`

Une fonction doit contenir l'instruction `Retourner`, et ce sur tous les chemins d'exécution possibles. Si tu écris un `Si ... Alors Retourner a FinSi` sans branche `Sinon`, la fonction ne retourne rien quand la condition est fausse : le résultat est alors imprévisible. Autre piège : écrire `Ecrire x * x` au lieu de `Retourner x * x`. `Ecrire` montre une valeur à l'écran ; `Retourner` la transmet à l'algorithme appelant pour qu'il puisse s'en servir dans un calcul.

5.3 La procédure : un sous-programme qui agit

DÉFINITION Procédure

Une **procédure** est un sous-programme qui reçoit éventuellement des paramètres et effectue une action (affichage, modification de données, tri...), mais qui ne retourne aucune valeur. Elle ne contient donc pas d'instruction `Retourner` de résultat.

PROPRIÉTÉ Critère de distinction fonction / procédure

Pour choisir entre fonction et procédure, pose-toi une seule question : le sous-programme doit-il produire une valeur que l'appelant réutilisera dans un calcul ?

- Oui (un seul résultat) $\Rightarrow$ c'est une fonction, avec `Retourner`.
- Non, il se contente d'agir (afficher, ranger, modifier) $\Rightarrow$ c'est une procédure.

EXEMPLE Une procédure d'affichage de tableau

Grâce au type tableau étudié à la section précédente, on peut passer un tableau entier en paramètre :

```
Procédure AfficherTableau(T : Tableau[1..n] de Entier ; n : Entier)
Variable
  i : Entier
Debut
  Pour i De 1 A n Faire
    Ecrire("Element", i, " : ", T[i])
  FinPour
Fin
```

Cette procédure agit (elle affiche les éléments du tableau) mais ne retourne rien : on ne peut donc pas écrire `x ← AfficherTableau(T, 5)`.

5.4 Appeler une fonction ou une procédure

Déclarer un sous-programme ne l'exécute pas : il faut l'appeler. La manière d'appeler diffère selon la nature du sous-programme.

PROPRIÉTÉ Les deux formes d'appel

- Une fonction s'appelle dans une expression, car elle produit une valeur : `y ← Carre(5)`, `Ecrire(Max(a, b))`, ou même `z ← Carre(3) + Max(2, 7)`.
- Une procédure s'appelle comme une instruction à part entière, sur sa propre ligne : `AfficherTableau(Notes, 30)`.

ATTENTION Ne pas confondre les deux appels

Écrire `Carre(5)` seul sur une ligne, sans récupérer le résultat, fait perdre la valeur calculée : c'est un appel inutile. À l'inverse, écrire `x ← AfficherTableau(T, n)` est une erreur : une procédure ne retourne rien, il n'y a rien à ranger dans `x`.

5.5 Paramètres formels et paramètres effectifs

DÉFINITION Paramètres formels et effectifs

Les **paramètres formels** sont les noms déclarés dans l'en-tête du sous-programme : ce sont des « places réservées » (ex. `x` dans `Fonction Carre(x : Reel)`). Les **paramètres effectifs** (ou arguments) sont les valeurs concrètes fournies lors de l'appel (ex. `5` dans `Carre(5)`). À l'appel, chaque paramètre effectif est associé au paramètre formel de même position : leur nombre, leur ordre et leur type doivent correspondre.

Ainsi, dans `Max(Note1, Note2)`, `a` reçoit la valeur de `Note1` et `b` celle de `Note2`. Les noms peuvent être totalement différents : seule la position compte.

EXERCICE RÉSOLU 5 La fonction Facture du cybercafé

Le cybercafé « Espace Numérique » de Bafoussam facture 500 FCFA l'heure de connexion. Pour fidéliser sa clientèle, le gérant accorde une réduction de 20 % sur le montant total dès que la durée dépasse 2 heures. Écrire la fonction `Facture(duree)` puis détailler son exécution pour une durée de 3 heures.

SOLUTION

La fonction :

```

Fonction Facture(duree : Reel) : Reel
Variable
    montant : Reel
Debut
    montant ← duree * 500
    Si duree > 2 Alors
        montant ← montant * 0.8 // reduction de 20 \%
    FinSi
    Retourner montant
Fin
  
```

Exécution de l'appel `prix ← Facture(3)` :

1. Le paramètre formel `duree` reçoit la valeur 3.

2. $\text{montant} \leftarrow 3 * 500$, donc montant vaut 1500.

3. La condition $\text{duree} > 2$ est vraie ($3 > 2$) : $\text{montant} \leftarrow 1500 * 0.8 = 1200$.

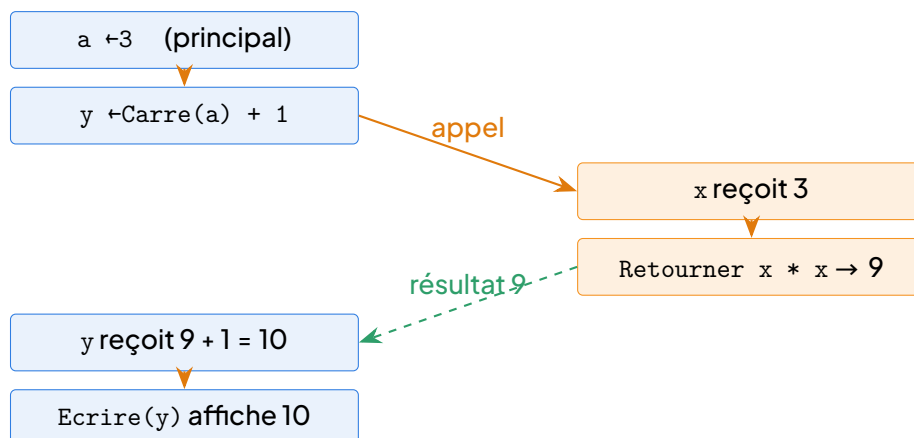
4. Retourner montant : la fonction rend 1200.

5. Dans l'algorithme principal, prix reçoit la valeur 1200.

Pour 3 heures, le client paie donc 1200 FCFA au lieu de 1500 FCFA. Si l'on avait appelé $\text{Facture}(1, 5)$, la condition aurait été fausse et le résultat aurait été 750 FCFA, sans réduction.

5.6 Exécution pas à pas : le flux d'appel

Que se passe-t-il exactement quand l'algorithme principal appelle une fonction ? L'exécution du principal est suspendue, la fonction s'exécute entièrement, puis le résultat revient au point d'appel et le principal reprend là où il s'était arrêté.



Flux d'appel : l'algorithme principal (bleu) appelle la fonction `Carre` (orange), qui retourne son résultat (flèche verte pointillée) ; le principal reprend ensuite son exécution.

EXPÉRIENCE Tracer un appel de fonction à la main

Objectif : vérifier sa compréhension du mécanisme d'appel en exécutant « sur papier » un algorithme complet.

Fonction `Max(a : Reel ; b : Reel) : Reel`

Debut

Si $a \geq b$ Alors

Retourner `a`

Sinon

Retourner `b`

```

    FinSi
Fin

Algorithme Principal
Variable
    n1, n2, m : Reel
Debut
    n1 ← 12,5
    n2 ← 14
    m ← Max(n1, n2)
    Ecrire("La meilleure note est : ", m)
Fin

```

Manipulation : construis un tableau d'exécution avec trois colonnes (ligne exécutée, valeur de $n1/n2/m$, valeur de a/b dans la fonction). Observations attendues : à l'appel, a vaut 12,5 et b vaut 14 ; la condition $a \geq b$ est fausse, donc la branche *Si non* retourne 14 ; m reçoit 14 et le message affiché est « La meilleure note est : 14 ». Interprétation : les variables a et b n'existent que pendant l'exécution de la fonction ; elles disparaissent dès le retour au programme principal.

5.7 Variables locales et variables globales

L'exemple précédent montre que les paramètres a et b de la fonction *Max* ne sont accessibles qu'à l'intérieur de cette fonction. C'est le concept de **portée** (ou visibilité) des variables.

DÉFINITION Variable locale

Une **variable locale** est déclarée à l'intérieur d'un sous-programme (dans sa section *Variable*). Elle n'existe que pendant l'exécution de ce sous-programme. À la fin du sous-programme, elle est détruite et sa valeur est perdue. Deux sous-programmes différents peuvent utiliser le même nom pour des variables locales distinctes sans aucun conflit.

DÉFINITION Variable globale

Une **variable globale** est déclarée dans l'algorithme principal (ou en dehors de tout sous-programme). Elle est accessible partout : dans le principal, dans les fonctions, dans les procédures. Sa valeur persiste pendant toute l'exécution du programme.

PROPRIÉTÉ Règle de priorité (masquage)

Si une variable locale (ou un paramètre) porte le même nom qu'une variable globale, c'est

la variable locale qui est utilisée à l'intérieur du sous-programme. La variable globale est « masquée » (inaccessible) tant que le sous-programme s'exécute.

EXEMPLE Variable locale vs variable globale

```

Variable
  x : Entier // x est GLOBALE

Fonction Test() : Entier
  Variable
    x : Entier // x est LOCALE à Test (masque la globale)
  Debut
    x ← 10
    Retourner x
  Fin

Algorithme Principal
  Debut
    x ← 5
    Ecrire("Avant appel : x = ", x) // Affiche 5
    Ecrire("Dans Test : ", Test()) // Affiche 10 (x locale)
    Ecrire("Après appel : x = ", x) // Affiche 5 (globale inchangée)
  Fin
  
```

ATTENTION Danger des variables globales

Modifier une variable globale à l'intérieur d'une fonction ou d'une procédure (effet de bord) rend le programme difficile à lire et à déboguer : le sous-programme change l'état du programme principal « en cachette ». Bonne pratique : privilégier le passage par paramètres et le retour de valeur (fonction) plutôt que l'utilisation de variables globales pour communiquer des résultats.

LESAVAIS-TU ? Et dans les vrais langages ?

En langage C, que tu étudieras cette année, la distinction fonction/procédure existe aussi : une « procédure » y est une fonction dont le type de retour est `void` (vide). Les variables déclarées dans un bloc `{ ... }` sont locales à ce bloc. En Python, la distinction disparaît presque : une procédure est simplement une fonction qui ne retourne rien — elle retourne en réalité une valeur spéciale appelée `None`. Quel que soit le langage, l'idée reste la même : nommer un traitement pour le réutiliser et limiter la portée des variables pour éviter les in-

terférences.

EXERCICE 5 À toi de jouer

1. Écris une fonction `PerimetreCercle(r : Reel) : Reel` qui retourne $2 \times 3,14 \times r$, puis un algorithme principal qui l'appelle pour un cercle de rayon 10 cm.
2. Écris une procédure `AfficherPairs(n : Entier)` qui affiche tous les nombres pairs de 2 à n .
3. Pour chaque sous-programme suivant, dis s'il doit être une fonction ou une procédure, et justifie : (a) calculer la moyenne d'un élève ; (b) afficher un bulletin ; (c) tester si un nombre est premier.
4. Soit l'algorithme ci-dessous. Que affiche-t-il ? Justifie en distinguant les variables locales et globales.

```

Variable
  a : Entier
Fonction F(x : Entier) : Entier
  Variable
    a : Entier
  Debut
    a ← x + 2
    Retourner a
  Fin
Algorithme Principal
  Debut
    a ← 10
    Ecrire(F(5))
    Ecrire(a)
  Fin
  
```

CORRIGÉ Exercice

1. Fonction `PerimetreCercle(r : Reel) : Reel` ; **corps** : `Retourner 2 * 3.14 * r`. Dans le principal : `p ← PerimetreCercle(10)` donne $p = 62,8$ cm.
2. Procédure avec une boucle : Pour i De 2 A n Pas 2 Faire `Ecrire(i)` FinPour (ou avec un `Si i mod 2 = 0` si Pas n n'est pas autorisé).
3. (a) Fonction : elle produit une valeur réutilisable (la moyenne). (b) Procédure : elle agit (affichage) sans rien retourner. (c) Fonction retournant un booléen : le résultat (vrai/faux) sert dans des tests conditionnels.

4. Affichage : 7 puis 10. Explication : Dans `F`, le paramètre `x` vaut 5. La variable `a` déclarée dans `F` est locale : elle masque la globale. `a` (locale) devient 7 et est retournée. La variable `a` globale (dans le principal) n'a jamais été modifiée, elle vaut toujours 10.

À RETENIR L'essentiel de la section

Une **fonction** calcule et `Retourner` un unique résultat ; elle s'appelle dans une expression. Une **procédure** effectue une action sans retourner de valeur ; elle s'appelle comme une instruction. Les **paramètres formels** (déclaration) reçoivent les **paramètres effectifs** (appel) selon leur position. Une **variable locale** (déclarée dans le sous-programme) n'existe que pendant son exécution ; une **variable globale** (déclarée dans le principal) est visible partout mais source d'erreurs si elle est modifiée dans un sous-programme. Lors d'un appel, le programme principal se suspend, le sous-programme s'exécute, puis le résultat revient au point d'appel. Découper ses programmes, c'est les rendre plus lisibles, plus fiables et plus faciles à corriger.

6 PARAMÈTRES, VARIABLES LOCALES ET GLOBALES

Dans la section précédente, nous avons vu comment structurer un algorithme en sous-programmes : les **fonctions** (qui renvoient une valeur) et les **procédures** (qui effectuent une action). Mais une question cruciale se pose : comment ces sous-programmes communiquent-ils avec le reste du programme ? Comment une procédure `Trier` peut-elle modifier le tableau du programme principal ? Comment une fonction `CalculerMoyenne` reçoit-elle les notes ? C'est le rôle des **paramètres** et de la **portée des variables** (locales vs globales). Maîtriser ces concepts, c'est éviter les bugs invisibles qui font perdre des heures de débogage — une compétence attendue au Probatoire et indispensable pour tout développeur.

6.1 Les deux modes de passage des paramètres

Lorsqu'on appelle une procédure ou une fonction, on lui transmet des **arguments** (valeurs ou variables). Le sous-programme les reçoit dans ses **paramètres formels**. Il existe deux mécanismes fondamentaux : le passage par valeur et le passage par référence.

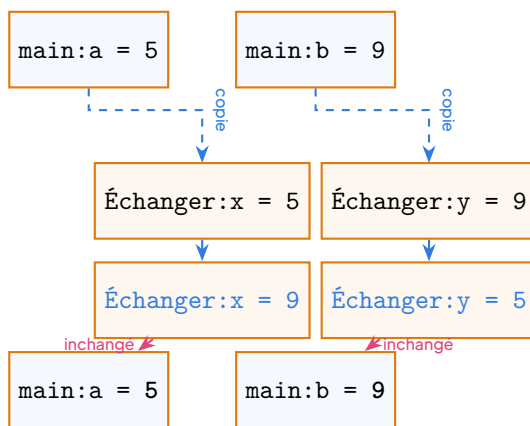
DÉFINITION Passage de paramètre par valeur

Dans le passage par valeur, le paramètre formel reçoit une copie de la valeur de l'argument. Toute modification du paramètre à l'intérieur du sous-programme n'affecte pas la variable d'origine dans l'appelant. C'est le mode par défaut dans la plupart des langages (C, Java, Python pour les types immuables, JavaScript pour les primitives) et dans notre pseudo-code algorithmique si rien n'est précisé.

DÉFINITION Passage de paramètre par référence (ou par variable)

Dans le passage par référence, le paramètre formel devient un alias (un autre nom) de la variable d'origine. Il n'y a pas de copie : le paramètre et l'argument désignent la même case mémoire. Toute modification du paramètre modifie immédiatement la variable de l'appelant. En pseudo-code, on le signale souvent par le mot-clé **Var** devant le paramètre. En C, on simule ce passage en transmettant l'adresse de la variable (pointeur).

Passage par VALEUR : Copie



Passage par REFERENCE : Alias

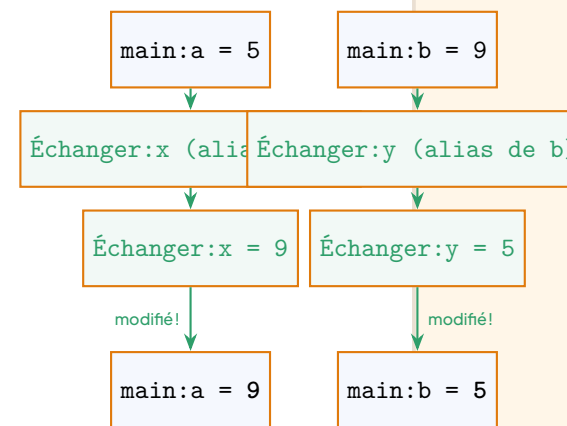


Schéma mémoire : passage par valeur (copie, variables locales indépendantes) vs passage par référence (alias, même case mémoire).

PROPRIÉTÉ Identification du mode de passage en pseudo-code

Dans l'en-tête d'une procédure ou fonction :

- Sans mot-clé (ex: Procédure Échanger(x, y : Entier)) → Passage par valeur.
- Avec mot-clé **Var** (ex: Procédure Échanger(Var x, y : Entier)) → Passage par référence.

Règle de décision : Si le sous-programme doit modifier la variable de l'appelant (ex : trier un tableau, échanger deux variables, saisir une valeur), on utilise le passage par référence (Var). Sinon (simple lecture, calcul), on préfère le passage par valeur (plus sûr, évite les effets de bord).

6.2 Variables locales et variables globales : la portée

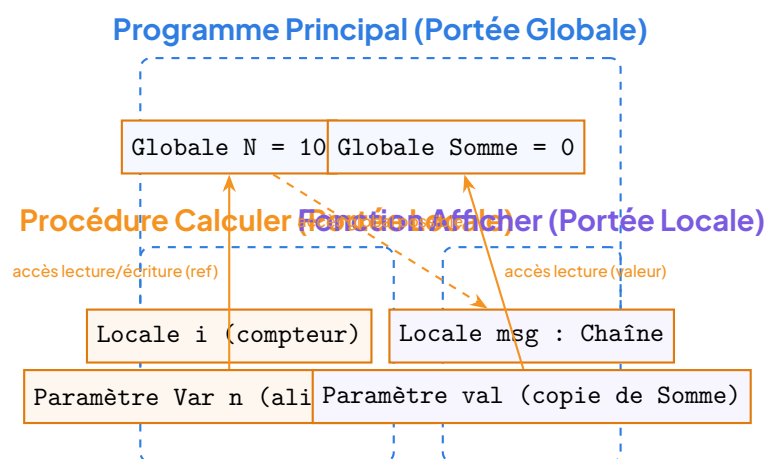
Où une variable « existe-t-elle » ? Qui peut la lire ou la modifier ? C'est la notion de **portée** (ou scope).

DÉFINITION Variable locale

Une variable locale est déclarée à l'intérieur d'une fonction ou d'une procédure (dans sa section `Variable` ou `Local`). Elle n'est visible et accessible que dans ce bloc. Elle est créée à l'entrée du sous-programme et détruite à sa sortie. Deux sous-programmes différents peuvent avoir des variables locales du même nom sans conflit : ce sont des cases mémoire distinctes.

DÉFINITION Variable globale

Une variable globale est déclarée dans le programme principal (souvent avant la déclaration des sous-programmes). Elle est visible et accessible partout : dans le programme principal, dans toutes les procédures et toutes les fonctions. Elle existe pendant toute l'exécution du programme.



Organisation de la mémoire : variables globales accessibles partout, variables locales cloisonnées dans leur sous-programme. Les paramètres par référence (Var) pointent vers la globale ; les paramètres par valeur en ont une copie locale.

À RETENIR Règle de masquage (ombre)

Si une variable locale (ou un paramètre) porte le même nom qu'une variable globale, la variable locale masque la globale à l'intérieur du sous-programme. Toute référence à ce nom désigne la locale. La globale reste inchangée et redevient accessible à la sortie du sous-programme. Évitez cette pratique : elle rend le code illisible et source d'erreurs.

6.3 Exécution guidée : la procédure Échanger — trace complète

C'est l'exemple classique du Probatoire pour tester la compréhension des passages de paramètres. Étudions les deux versions.

EXEMPLE Algorithme : Échange de deux variables — Version PAR VALEUR (ÉCHEC)

```

Algorithme EchangeValeur
Variable
  a, b : Entier
Début
  a ← 5
  b ← 9
  Afficher("Avant ÉchangerV : a=", a, " b=", b) // Affiche 5 et 9
  ÉchangerV(a, b) // Passage par valeur (pas de Var)
  Afficher("Après ÉchangerV : a=", a, " b=", b) // Affiche 5 et 9 (INCHANGÉ !)
Fin

Procédure ÉchangerV(x, y : Entier) // SANS Var → Par valeur
Variable
  temp : Entier
Début
  temp ← x
  x ← y
  y ← temp
  // x et y sont des copies locales, a et b ne bougent pas
Fin

```

Version incorrecte pour échanger les variables de l'appelant.

EXEMPLE Algorithme : Échange de deux variables — Version PAR REFERENCE (SUCCÈS)

```

Algorithme EchangeReference
Variable
  a, b : Entier
Début
  a ← 5
  b ← 9
  Afficher("Avant ÉchangerR : a=", a, " b=", b) // Affiche 5 et 9
  ÉchangerR(a, b) // Passage par référence grâce à Var
  Afficher("Après ÉchangerR : a=", a, " b=", b) // Affiche 9 et 5 (MODIFIÉ !)
Fin

Procédure ÉchangerR(Var x, y : Entier) // AVEC Var → Par référence

```

```

Variable
  temp : Entier
Début
  temp <- x // temp = valeur de a (5)
  x <- y    // x (alias de a) prend valeur de b (9) -> a devient 9
  y <- temp // y (alias de b) prend temp (5) -> b devient 5
Fin

```

Version correcte : le mot-clé Var change tout.

Étape	Variable a (Main)	Variable b (Main)	Variable x (Proc)	Variable y (Proc)
PASSAGE PAR VALEUR (Échec)				
Init Main	5	9	-	-
Appel ÉchangerV(5, 9)	5	9	5 (copie)	9 (copie)
temp <- x	5	9	5	9
x <- y	5	9	9	9
y <- temp	5	9	9	5
Fin Procédure	5	9	détruites	détruites
PASSAGE PAR REFERENCE (Succès)				
Init Main	5	9	-	-
Appel ÉchangerR(a, b)	5	9	alias de a	alias de b
temp <- x	5	9	5	9
x <- y	9	9	9	9
y <- temp	9	5	9	5
Fin Procédure	9	5	détruites	détruites

Tableau de trace détaillé : comparaison pas à pas des deux modes de passage sur l'algorithme Échanger.

6.4 Bonnes pratiques et pièges à éviter

MÉTHODE

Comment bien gérer paramètres et portées

1. Privilégiez le passage par valeur par défaut. C'est plus sûr : le sous-programme ne peut pas casser l'état de l'appelant par surprise.
2. Utilisez le passage par référence (Var) uniquement si nécessaire : pour modifier une variable de l'appelant (résultat de procédure, saisie, tri de tableau, échange).

Documentez-le : `// Var : modifié par la proc.`

3. Évitez les variables globales autant que possible. Elles créent un couplage fort : n'importe quelle procédure peut les modifier n'importe quand, rendant le débogage cauchemardesque. Passez les données par paramètres.
4. Nommez distinctement vos variables locales et globales (ex : préfixe `g_` pour globales, ou noms explicites `compteurBoucle` vs `totalGeneral`).
5. Initialisez vos variables locales à chaque entrée de procédure/fonction. Une locale non initialisée contient une valeur aléatoire (garbage value).

ATTENTION Erreur classique au Probatoire : « Ma procédure ne marche pas ! »

- Symptôme : Vous appelez `TriTableau(T)`, le tableau `T` n'est pas trié au retour.
- Cause : Vous avez oublié `Var` dans l'en-tête : Procédure `TriTableau(T : Tableau)` au lieu de Procédure `TriTableau(Var T : Tableau)`. Le tri s'est fait sur une copie locale du tableau, détruite à la fin.
- Autre piège : Croire que Fonction `F(x : Entier) : Entier` avec `x <- x + 1` modifie l'argument. Faux : `x` est local (passage par valeur). Une fonction ne devrait jamais modifier ses arguments (effet de bord) ; c'est le rôle d'une procédure `Var`.

LESAVAIS-TU ? Les variables globales : fléau des grands logiciels

Dans les années 70–80, les programmes en Fortran ou Basic utilisaient massivement des variables globales (`COMMON_blocks`). Résultat : le « spaghetti code » où une modification dans un module cassait un module à l'autre bout du programme. L'ingénierie logicielle moderne (C, Ada, Java, Python, C++) impose l'encapsulation : données privées (locales), communication contrôlée par paramètres. Au Cameroun, les systèmes d'information de la fonction publique (paie, gestion des élèves) migrent vers des architectures modulaires pour éviter ces bugs « fantômes » liés aux globales mal maîtrisées.

6.5 Exercice résumé : trace d'exécution complète

EXERCICE RÉSOLU 6 Trace avec variables locales, globales et paramètres mixtes

Énoncé : Exécuter l'algorithme suivant et donner l'affichage final.

Algorithme `TraceGlobaleLocale`

```

Variable
  X, Y : Entier // Globales
Début
  X <- 10
  Y <- 20
  Proc1(X, Y)    // Appel 1
  Afficher("Après Proc1 : X=", X, " Y=", Y)
  Proc2(Y)       // Appel 2
  Afficher("Après Proc2 : X=", X, " Y=", Y)
Fin

Procédure Proc1(Var A, B : Entier) // A et B par référence
Variable
  X : Entier // Locale à Proc1 (masque la globale X)
Début
  X <- 5      // Modifie la LOCALE X
  A <- A + X   // A (alias de globale X) = 10 + 5 = 15
  B <- B + 1   // B (alias de globale Y) = 20 + 1 = 21
  Afficher("Dans Proc1 : X(loc)=", X, " A=", A, " B=", B)
Fin

Procédure Proc2(Var C : Entier) // C par référence
Variable
  Y : Entier // Locale à Proc2 (masque la globale Y)
Début
  Y <- 100
  C <- C + Y   // C (alias de globale Y) = 21 + 100 = 121
Fin

```

Solution détaillée :

1. Initialisation Main : X=10, Y=20.
2. Appel Proc1(X, Y) : A alias de X(glob), B alias de Y(glob). X(loc) déclarée.
 - $X(loc) \leftarrow 5$
 - $A \leftarrow A + X(loc) \rightarrow X(glob) = 10 + 5 = 15.$
 - $B \leftarrow B + 1 \rightarrow Y(glob) = 20 + 1 = 21.$
 - Affichage Proc1 : X(loc)=5, A=15, B=21.
3. Retour Main : X(glob)=15, Y(glob)=21. X(loc) détruite. Affichage Main : Après Proc1 : X=15 Y=21.
4. Appel Proc2(Y) : C alias de Y(glob) (valeur 21). Y(loc) déclarée.
 - $Y(loc) \leftarrow 100.$
 - $C \leftarrow C + Y(loc) \rightarrow Y(glob) = 21 + 100 = 121.$
5. Retour Main : X(glob)=15, Y(glob)=121. Affichage final : Après Proc2 : X=15 Y=121.

Résultat affiché :

Dans Proc1 : X(loc)=5 A=15 B=21

Après Proc1 : X=15 Y=21

Après Proc2 : X=15 Y=121

6.6 Exercices d'entraînement

EXERCICE 6 Identification de passage et trace

Soit l'algorithme suivant :

```
Algorithme TestParam
Variable a, b : Entier
Début
    a <- 3; b <- 7
    P1(a, b)
    Afficher("1: a=", a, " b=", b)
    P2(a, b)
    Afficher("2: a=", a, " b=", b)
Fin

Procédure P1(u, v : Entier) // (1)
Début
    u <- u + 10
    v <- v * 2
Fin

Procédure P2(Var u, v : Entier) // (2)
Début
    u <- u + 10
    v <- v * 2
Fin
```

Questions :

1. Quel est le mode de passage de u et v dans $P1$? Dans $P2$?
2. Donner l'affichage complet de l'algorithme.
3. Expliquer pourquoi les résultats diffèrent.

CORRIGÉ Exercice n°1

1. $P1$: passage par valeur (pas de `Var`). $P2$: passage par référence (mot-clé `Var`).

2. Trace :

- **Début** : $a=3$, $b=7$.
- **P1(a,b)** : u (copie de $a=3$) devient 13. v (copie de $b=7$) devient 14. a, b inchangés (3, 7).
- **Affichage 1** : 1: $a=3$ $b=7$.
- **P2(a,b)** : u alias de a . $a = 3+10 = 13$. v alias de b . $b = 7*2 = 14$.
- **Affichage 2** : 2: $a=13$ $b=14$.

3. Dans P1, on modifie des copies locales détruites à la fin. Dans P2, u et v sont a et b : la modification persiste.

EXERCICE 7 Portée et masquage

Compléter le tableau de trace pour l'algorithme ci-dessous et indiquer l'affichage final.

```

Algorithme Masquage
Variable X : Entier // Globale
Début
    X ← - 5
    F1()
    Afficher("Main 1 : X=", X)
    F2()
    Afficher("Main 2 : X=", X)
Fin

Procédure F1()
Variable X : Entier // Locale à F1
Début
    X ← - 10
    Afficher("F1 : X=", X)
Fin

Procédure F2()
Variable Y : Entier // Locale à F2
Début
    Y ← - 20
    X ← - X + Y // Quel X ? Quel Y ?
    Afficher("F2 : X=", X, " Y=", Y)
Fin
  
```

CORRIGÉ Exercice n°2

- **Main** : $X(\text{glob})=5$.
- **F1()** : Déclare $X(\text{loc})$. $X(\text{loc})=10$. $X(\text{glob})$ inchangé (5). Affiche F1: $X=10$.
- **Retour Main** : $X(\text{glob})=5$. Affiche Main 1: $X=5$.
- **F2()** : Déclare $Y(\text{loc})=20$. Instruction $X \leftarrow X + Y$:
 - **Y : locale** $Y(\text{loc})=20$.
 - **X (à gauche et à droite)** : pas de X locale dans F2, donc c'est la globale $X(\text{glob})=5$.
 $X(\text{glob}) = 5 + 20 = 25$. Affiche F2: $X=25$ $Y=20$.
- **Retour Main** : $X(\text{glob})=25$. Affiche Main 2: $X=25$.

Affichage final :

F1: $X=10$

Main 1: $X=5$

F2: $X=25$ $Y=20$

Main 2: $X=25$

À RETENIR Check-list pour le Probatoire – Section Algorithmique

- Var dans l'en-tête $\Leftrightarrow$ Passage par référence (modification possible de l'argument).
- Pas de Var $\Leftrightarrow$ Passage par valeur (copie, argument protégé).
- Variable locale = déclarée dans le sous-programme, vie limitée à l'exécution de ce sous-programme.
- Variable globale = déclarée dans le principal, vie = tout le programme, accessible partout (danger!).
- Masquage : Locale masque Globale si même nom. À éviter.
- Tableau : Toujours passé par référence (adresse de base) en pratique (C, pseudo-code souvent implicite), mais en pseudo-code strict : Var Tab : Tableau pour le modifier.
- Fonction : renvoie une valeur (Retourner), idéalement sans effet de bord (ne modifie pas ses paramètres). Procédure : action, peut modifier paramètres Var.

7 ACTIVITÉ D'INTÉGRATION

7.1 Objectifs de l'activité

À l'issue de cette activité d'intégration, tu seras capable de :

- concevoir un algorithme **modulaire** complet, c'est-à-dire décomposé en un programme principal, une **fonction** et une **procédure** ;
- mobiliser simultanément les trois **structures de contrôle** (séquentielle, conditionnelle, itérative) dans un même problème ;
- déclarer, remplir et parcourir un **tableau** à une dimension (lecture, calculs cumulés, recherche d'un maximum) ;
- identifier le **mode de passage des paramètres** (par valeur ou par référence) et distinguer **variable locale** et **variable globale** ;
- **exécuter à la main** un algorithme sur un jeu de données réduit et vérifier la cohérence des résultats obtenus.

7.2 Situation

Monsieur Etoundi est le gérant du cybercafé « Le Connecté », situé non loin du lycée de Nkong-samba. Chaque soir, il consigne dans un cahier les durées de connexion (en minutes) de ses clients, puis calcule laborieusement la recette de la journée à l'aide d'une calculatrice. Fatigué par ces calculs répétitifs — et par les erreurs qu'ils provoquent parfois —, il sollicite les élèves de la classe de Première pour concevoir un algorithme qui fera ce travail automatiquement.

La grille tarifaire du cybercafé est la suivante :

- les **30 premières minutes** coûtent un forfait de **100 FCFA** ;
- au-delà, chaque **tranche de 15 minutes** entamée coûte **25 FCFA** supplémentaires ;
- toute connexion dont la durée **dépasse strictement 120 minutes** bénéficie d'une **réduction de 10 %** sur le montant total.

Pour tester l'algorithme, M. Etoundi fournit les durées de connexion de ses **10 clients** du jour, qu'il souhaite stocker dans un tableau T de 10 cases :

Client n°	1	2	3	4	5	6	7	8	9	10
Durée (min)	20	45	30	90	125	15	60	150	40	100

M. Etoundi souhaite que l'algorithme affiche, à la fin de la journée : la **recette totale** du jour, la **durée moyenne** de connexion, ainsi que le **numéro du client** resté connecté le plus longtemps.

7.3 Consignes

Travail en groupes de 4 élèves. Chaque groupe rédige sa solution sur papier, puis tous les groupes comparent leurs résultats.

1. **Analyse du tarif.** À la main, calcule le montant dû pour une connexion de 20 minutes, puis de 45 minutes, puis de 125 minutes. Déduis-en la règle générale de calcul.
2. **La fonction.** Écris en pseudo-code la fonction `Montant(duree)` qui reçoit une durée en minutes et retourne le prix à payer. Précise le type du paramètre et le type de la valeur retournée. Le paramètre `duree` doit-il être passé par valeur ou par référence? Justifie.
3. **La procédure.** Écris la procédure `Saisir(T, n)` qui remplit le tableau `T` de `n` durées saisies au clavier, en refusant toute durée négative (boucle de contrôle de saisie). Le tableau `T` doit-il être passé par valeur ou par référence? Justifie.
4. **L'algorithme principal.** Écris l'algorithme `Caisse` qui : déclare le tableau, appelle `Saisir`, puis à l'aide d'une boucle `Pour` calcule la recette totale (en appelant `Montant` pour chaque case), la durée moyenne, et recherche le numéro du client ayant la plus longue connexion.
5. **Exécution à la main.** Construis le tableau d'exécution (trace) de ton algorithme sur les 10 durées fournies : pour chaque client, indique la durée, le montant, la recette cumulée, et l'évolution du maximum.
6. **Vérification.** Compare tes résultats (recette totale, moyenne, client champion) avec ceux des autres groupes. En cas de désaccord, repérez ensemble la case du tableau où les traces divergent.
7. **Extension.** Dans la procédure `Saisir`, que se passerait-il si le tableau `T` était passé par valeur? Identifie précisément quels paramètres de ton algorithme exigent un passage par référence.

7.4 Ressources à mobiliser

À RETENIR

Boîte à outils de l'activité

- **Boucle bornée :** Pour $i \leftarrow 1$ à n Faire ... FinPour ; **boucle de contrôle :** TantQue (condition) Faire ... FinTantQue.
- **Tableau :** déclaration `T : tableau[1..10] de Entier` ; **accès à une case :** `T[i]`.
- **Fonction :** retourne une valeur via `Retourner` ; **procédure :** effectue une action, ne retourne rien.
- **Passage par valeur :** le sous-programme travaille sur une copie (la donnée d'origine est protégée). **Passage par référence :** le sous-programme modifie directement la variable de l'appelant.
- **Techniques de parcours :** accumulateur ($\text{somme} \leftarrow \text{somme} + T[i]$), recherche de maximum (mémoriser la valeur et l'indice du plus grand élément rencontré).
- **Tranche entamée :** le nombre de tranches de 15 minutes au-delà de 30 minutes est $\left\lceil \frac{d-30}{15} \right\rceil$ (division entière arrondie au supérieur).

7.5 Démarche et corrigé commenté

EXERCICE RÉSOLU 7 La caisse du cybercafé « Le Connecté »

Énoncé : concevoir l'algorithme modulaire de gestion de la caisse, puis l'exécuter à la main sur les durées : 20, 45, 30, 90, 125, 15, 60, 150, 40, 100.

SOLUTION

Étape 1 — Comprendre le tarif. Pour 20 min (≤ 30) : forfait de 100 FCFA. Pour 45 min : on dépasse 30 min de 15 min, soit 1 tranche : $100 + 1 \times 25 = 125$ FCFA. Pour 125 min : dépassement de 95 min, soit $\lceil 95/15 \rceil = 7$ tranches : $100 + 7 \times 25 = 275$ FCFA ; comme $125 > 120$, réduction de 10 % : $275 \times 0,9 = 247,5$ FCFA.

Étape 2 — La fonction. Le paramètre *duree* n'est pas modifié par le calcul : passage par valeur.

```
Fonction Montant(duree : Entier) : Reel
Variable prix : Reel
Debut
    Si duree <= 30 Alors
        prix <- 100
    Sinon
        // tranches entamees au-dela de 30 min
        prix <- 100 + 25 * ((duree - 30 + 14) DIV 15)
    FinSi
    Si duree > 120 Alors
        prix <- prix * 0.9 // reduction de 10 %
    FinSi
    Retourner prix
Fin
```

L'astuce $(duree - 30 + 14) \text{ DIV } 15$ réalise l'arrondi au supérieur avec une division entière : pour 45 min, $(15 + 14) \div 15 = 1$; pour 46 min, $(16 + 14) \div 15 = 2$.

Étape 3 — La procédure. Le tableau doit revenir rempli à l'algorithme principal : passage par référence.

```
ProcEDURE Saisir(Var T : tableau[1..10] de Entier ; n : Entier)
Variable i : Entier
Debut
    Pour i <- 1 à n Faire
        Ecrire("Durée du client ", i, " : ")
        Lire(T[i])
        TantQue T[i] < 0 Faire // controle de saisie
            Ecrire("Durée invalide, recommencez : ")
            Lire(T[i])
        FinTantQue
    FinPour
Fin
```

Étape 4 — L'algorithme principal.

```

Algorithme Caisse
Variable T : tableau[1..10] de Entier
Variable i, numMax : Entier
Variable recette, moyenne, sommeDurees : Reel
Debut
  Saisir(T, 10)
  recette <- 0 ; sommeDurees <- 0
  numMax <- 1
  Pour i <- 1 à 10 Faire
    recette <- recette + Montant(T[i])
    sommeDurees <- sommeDurees + T[i]
    Si T[i] > T[numMax] Alors
      numMax <- i
  FinSi
  FinPour
  moyenne <- sommeDurees / 10
  Ecrire("Recette totale : ", recette, " FCFA")
  Ecrire("Durée moyenne : ", moyenne, " min")
  Ecrire("Plus longue connexion : client ", numMax)
Fin

```

Ici, *recette*, *sommeDurees* et *numMax* sont des variables locales de l'algorithme principal ; *prix* et *i* de la fonction et de la procédure sont locales à leurs sous-programmes respectifs : aucune interférence.

Étape 5 — Trace d'exécution.

Client	Durée	Montant (FCFA)	Recette cumulée	numMax
1	20	100	100	1
2	45	125	225	2
3	30	100	325	2
4	90	$100 + 4 \times 25 = 200$	525	4
5	125	$275 \times 0,9 = 247,5$	772,5	5
6	15	100	872,5	5
7	60	$100 + 2 \times 25 = 150$	1022,5	5
8	150	$325 \times 0,9 = 292,5$	1315	8
9	40	$100 + 1 \times 25 = 125$	1440	8
10	100	$100 + 5 \times 25 = 225$	1665	8

Résultats affichés : recette totale = 1665 FCFA ; durée moyenne = $675/10 = 67,5$ min ; plus longue connexion = client n° 8 (150 min).

Étape 7 — Extension. Si *T* était passé par valeur à *Saisir*, la procédure remplirait une copie du tableau, détruite à la fin de la procédure : le tableau du programme principal resterait vide. Seul le paramètre *T* de *Saisir* exige le passage par référence (mot-clé *Var*) ; *n* et

duree, simples données lues, restent passés par valeur.

7.6 Bilan de l'activité

Cette activité a permis de mettre en évidence les points essentiels de la leçon :

- un problème réel se résout plus sûrement en le **décomposant en sous-programmes** : la fonction `Montant` isole la règle tarifaire (facile à modifier si M. Etoundi change ses prix), la procédure `Saisir` isole la saisie contrôlée ;
- la **boucle Pour** est l'outil naturel du parcours de tableau : accumulation (recette, somme), recherche de maximum avec mémorisation de l'indice ;
- le choix du **mode de passage des paramètres** n'est pas anodin : par valeur pour protéger une donnée, par référence pour qu'une procédure puisse modifier durablement une variable de l'appelant ;
- l'**exécution à la main** (trace) est une méthode de vérification irremplaçable : la comparaison des traces entre groupes a montré qu'une erreur se localise en retrouvant la première ligne où deux tableaux divergent ;
- enfin, les pièges classiques ont été repérés : oublier la réduction au-delà de 120 min, mal arrondir les tranches entamées, initialiser le maximum à 0 au lieu du premier élément, ou passer le tableau par valeur dans la procédure de saisie.

Tu disposes désormais de toutes les briques — structures de contrôle, tableaux, fonctions et procédures — pour attaquer la programmation en langage C, où ces notions se traduiront presque mot pour mot.

8 MÉTHODES ET EXERCICES RÉSOLUS

9 LEÇON 07 : ALGORITHMIQUE — STRUCTURES DE CONTRÔLE, FONCTIONS ET PROCÉDURES

9.1 Les structures itératives

DÉFINITION Structure itérative (Boucle)

Une **structure itérative** (ou boucle) permet de répéter un bloc d'instructions plusieurs fois. Trois formes existent en algorithmique standard (MINESEC) :

- **Pour** (borne) : nombre de répétitions connu à l'avance.
- **Tant que** (conditionnelle en entrée) : répétition tant qu'une condition est vraie ; test avant chaque tour (zéro tour possible).
- **Répéter ... Jusqu'à** (conditionnelle en sortie) : répétition jusqu'à ce qu'une condition devienne vraie ; test après chaque tour (au moins un tour garanti).

MÉTHODE Bien choisir sa boucle

Pour répéter un traitement, pose-toi ces trois questions dans l'ordre :

1. Connais-tu à l'avance le nombre exact de répétitions ?
⇒ Utilise **Pour** *i* de 1 à *n* **Faire** ... **FinPour**.
Cas fréquents : somme des *n* premiers entiers, factorielle, saisie de *n* notes, parcours d'un tableau.
2. Le nombre de tours est inconnu, mais la répétition dépend d'une condition à vérifier avant d'agir ?
⇒ Utilise **Tant que** (condition) **Faire** ... **FinTantQue**.
Le corps peut ne jamais s'exécuter si la condition est fausse dès le départ.
3. Le traitement doit s'exécuter au moins une fois (ex. saisie utilisateur), puis se répéter jusqu'à validité ?
⇒ Utilise **Répéter ... Jusqu'à** (condition).
Idéal pour une saisie contrôlée (ex. demander une note jusqu'à ce qu'elle soit entre 0 et 20).

Réflexes indispensables :

- Initialiser systématiquement les compteurs et accumulateurs avant la boucle ($s \leftarrow 0$,

$p \leftarrow 1, i \leftarrow 1$).

- Garantir la terminaison : dans un **Tant que**, au moins une variable intervenant dans la condition doit être modifiée dans le corps de la boucle.
- Valider par une table d'exécution : une colonne par variable, une ligne par tour de boucle (voir exemple résolu ci-dessous).

EXERCICE RÉSOLU 8 Somme des entiers pairs inférieurs ou égaux à n

Énoncé : Écrire un algorithme qui lit un entier naturel n ($n \geq 2$) et calcule la somme $S = 2 + 4 + 6 + \dots$ de tous les entiers pairs inférieurs ou égaux à n . Exécuter cet algorithme pour $n = 10$ en dressant la table d'exécution complète.

SOLUTION

Raisonnement. Le nombre de tours est connu si l'on parcourt les entiers de 1 à n : on utilise une boucle **Pour**. À chaque tour, on teste si l'entier courant i est pair ($i \bmod 2 = 0$) ; si oui, on l'ajoute à l'accumulateur S , initialisé à 0 avant la boucle.

```

Algorithme SommePaires
Variables n, i, S : Entier
Debut
    Ecrire("Entrer un entier n >= 2 : ")
    Lire(n)
    S ← 0 // Initialisation de l'accumulateur
    Pour i de 1 à n Faire
        Si (i mod 2 = 0) Alors // i est pair
            S ← S + i
        FinSi
    FinPour
    Ecrire("La somme des entiers pairs <= ", n, " est : ", S)
Fin
  
```

Table d'exécution pour $n = 10$ (on affiche l'état après chaque tour où S change) :

Itération (i)	i	i mod 2	S après le tour
Initialisation	—	—	0
1	1	1	0
2	2	0	$0 + 2 = 2$
3	3	1	2
4	4	0	$2 + 4 = 6$
5	5	1	6
6	6	0	$6 + 6 = 12$
7	7	1	12
8	8	0	$12 + 8 = 20$
9	9	1	20
10	10	0	$20 + 10 = 30$

Vérification mathématique : $S = 2 + 4 + 6 + 8 + 10 = 2(1 + 2 + 3 + 4 + 5) = 2 \times 15 = 30$. Conclusion : pour $n = 10$, l'algorithme affiche $S = 30$, ce qui confirme le résultat théorique.

9.2 Algorithmes sur les problèmes de mathématiques et de physique

MÉTHODE De l'énoncé scientifique à l'algorithme

1. Identifier les données (ce qui est lu : coefficients, vitesses, masses...) et les résultats (ce qui est affiché). En déduire les variables et leurs types (Reel pour les grandeurs physiques, Entier pour les compteurs).
2. Traduire les formules du cours en affectations : discriminant $\Delta = b^2 - 4ac$, énergie cinétique $E_c = \frac{1}{2}mv^2$, loi d'Ohm $U = RI$, etc.
3. Repérer les cas : chaque cas de l'énoncé (par exemple $\Delta < 0$, $\Delta = 0$, $\Delta > 0$) devient une branche d'une structure conditionnelle Si ... SinonSi ... Sinon ... FinSi.
4. Repérer les répétitions : un calcul répété (suite, somme, simulation pas à pas) devient une boucle (Pour ou Tant que).
5. Rédiger l'algorithme complet (en-tête, déclarations, corps entre Debut et Fin), puis tester sur un exemple numérique dont on connaît la réponse.

Formules usuelles à retenir :

- Équation $ax^2 + bx + c = 0$ ($a \neq 0$) : $\Delta = b^2 - 4ac$. Si $\Delta > 0$, $x_1 = \frac{-b-\sqrt{\Delta}}{2a}$, $x_2 = \frac{-b+\sqrt{\Delta}}{2a}$; si $\Delta = 0$, $x_0 = \frac{-b}{2a}$; si $\Delta < 0$, pas de solution réelle.
- Suite arithmétique : $u_n = u_0 + n \times r$. Suite géométrique : $u_n = u_0 \times q^n$.

- Chute libre : $h(t) = h_0 - \frac{1}{2}gt^2$, $v(t) = gt$.

EXERCICE RÉSOLU 9 Résolution de l'équation du second degré

Énoncé : Un élève de Première C veut automatiser la résolution dans $\mathbb{R}$ de l'équation $ax^2 + bx + c = 0$ ($a \neq 0$). Écrire l'algorithme correspondant, puis le tester avec $a = 1$, $b = -3$, $c = 2$.

SOLUTION

Raisonnement. Données : a, b, c (réels, $a \neq 0$). Résultat : les solutions réelles éventuelles. Le discriminant $\Delta = b^2 - 4ac$ commande trois cas : c'est une structure conditionnelle à trois branches.

```

Algorithme EquationSecondDegre
Variables a, b, c, Delta, x0, x1, x2 : Reel
Debut
    Ecrire("Entrer a (non nul), b et c : ")
    Lire(a, b, c)
    Delta <- b * b - 4 * a * c
    Si (Delta > 0) Alors
        x1 <- (-b - Racine(Delta)) / (2 * a)
        x2 <- (-b + Racine(Delta)) / (2 * a)
        Ecrire("Deux solutions : x1 = ", x1, " et x2 = ", x2)
    SinonSi (Delta = 0) Alors
        x0 <- -b / (2 * a)
        Ecrire("Solution double : x0 = ", x0)
    Sinon
        Ecrire("Pas de solution réelle")
    FinSi
Fin
  
```

Test avec $a = 1$, $b = -3$, $c = 2$:

$$\Delta = b^2 - 4ac = (-3)^2 - 4 \times 1 \times 2 = 9 - 8 = 1 > 0,$$

$$x_1 = \frac{-b - \sqrt{\Delta}}{2a} = \frac{3 - 1}{2} = 1,$$

$$x_2 = \frac{-b + \sqrt{\Delta}}{2a} = \frac{3 + 1}{2} = 2.$$

Vérification : $1^2 - 3 \times 1 + 2 = 0$ et $2^2 - 3 \times 2 + 2 = 0$. Conclusion : l'algorithme affiche « Deux solutions : $x_1 = 1$ et $x_2 = 2$ », ce qui est exact : le trinôme se factorise en $(x - 1)(x - 2)$.

9.3 Les structures de données : panorama

DÉFINITION Structure de données

Une **structure de données** est une organisation logique de plusieurs valeurs en mémoire pour permettre leur manipulation efficace (accès, insertion, suppression, parcours). Le choix dépend de la nature des données (homogènes/hétérogènes) et des opérations fréquentes.

MÉTHODE Reconnaître et choisir une structure de données

Pour répondre à une question « énumérer et définir », retiens ce tableau de référence :

Structure	Principe	Exemple d'usage camerounais
Tableau	Ensemble d'éléments de même type, accessibles par un indice entier (indexation directe). Taille fixe à la déclaration.	Les 30 notes d'informatique d'une classe de 1ère D ; le stock de 50 articles d'une boutique.
Enregistrement	Regroupe des champs de types différents sous un même nom pour décrire une entité complexe.	La fiche d'un élève : <code>nom</code> (chaîne), <code>âge</code> (entier), <code>moyenne</code> (réel), <code>redoublant</code> (booléen).
Pile	Structure LIFO (Last In, First Out) : le dernier élément ajouté (empilé) est le premier retiré (dépilé). Opérations : <code>Empiler</code> , <code>Dépiler</code> , <code>Sommet</code> .	Historique « Précédent » d'un navigateur web ; bouton « Annuler » (Ctrl+Z) dans Word ; gestion des appels de fonctions (pile d'exécution).
Liste	Suite d'éléments (souvent de même type) chaînés par des pointeurs/liens. Taille dynamique. Accès séquentiel. Opérations : <code>Inserer</code> , <code>Supprimer</code> , <code>Parcourir</code> .	Liste des clients connectés dans un cybercafé de Douala ; liste des processus actifs dans le gestionnaire de tâches.

Réflexe Probatoire : pour chaque structure citée, donne toujours une définition courte et un exemple concret contextualisé : c'est ce qui rapporte tous les points.

EXERCICE RÉSOLU 10 Choisir la bonne structure

Énoncé : Le secrétariat du Lycée de Nkongsamba veut mémoriser : (a) les 25 notes d'informatique d'une classe ; (b) pour chaque élève, son nom, sa date de naissance et sa moyenne ; (c) l'historique des pages visitées dans un navigateur (on revient en arrière

page par page). Pour chaque cas, indiquer la structure de données la plus adaptée et justifier.

SOLUTION

(a) Les 25 notes : un **tableau**. Les 25 valeurs sont toutes du même type (réel) et on accède à chacune par sa position (indice) : `Notes[7]` désigne la 7^e note. La taille est connue et fixe. Le tableau à une dimension est la structure naturelle.

(b) La fiche d'un élève : un **enregistrement**. Les informations sont de types différents (chaîne de caractères, date, réel) mais décrivent une même entité « élève ». L'enregistrement regroupe ces champs sous un seul identificateur :

Type Eleve = Enregistrement
nom : Chaîne
dateNaissance : Chaîne
moyenne : Reel
FinEnregistrement

(c) L'historique du navigateur : une **pile**. La dernière page visitée est la première à laquelle on revient avec le bouton « Précédent » : c'est exactement le comportement LIFO d'une pile, où l'on *Empile* chaque nouvelle page et on *Dépile* pour revenir en arrière.

Conclusion : (a) tableau (éléments homogènes, accès indexé); (b) enregistrement (champs hétérogènes, entité unique); (c) pile (accès en sens inverse de l'arrivée, LIFO).

9.4 Le type tableau à une dimension

DÉFINITION Tableau à une dimension

Un **tableau** (ou **vecteur**) est une collection finie et ordonnée de n éléments de même type, repérés par un indice variant de 1 à n (convention MINESEC). Déclaration : `Tableau T[1..n]` : Type. L'élément d'indice i s'écrit `T[i]`.

MÉTHODE Les quatre parcours classiques d'un tableau

Déclaration : `Tableau T[1..N]` : Reel (ou Entier, Chaîne, Boolean). L'indice varie de 1 à N .

1. Lecture (remplissage) : `boucle Pour avec Lire(T[i])` à l'intérieur.
2. Affichage : `boucle Pour avec Ecrire(T[i])`.
3. Recherche d'un maximum (ou minimum) : initialiser `max <- T[1]`, puis pour i de 2 à n : si `T[i] > max` alors `max <- T[i]`. Jamais initialiser le maximum à 0 (échec si toutes valeurs sont négatives).

4. Recherche d'une valeur (recherche séquentielle) : parcourir avec un booléen `trouve` $\leftarrow$ Faux; dès que $T[i] = \text{val}$, passer `trouve` à Vrai, mémoriser la position `pos` $\leftarrow$ i et sortir de la boucle (ou continuer pour compter les occurrences).

Piège mortel : ne jamais accéder à $T[0]$ ni à $T[n+1]$ si le tableau est déclaré $1..n$: c'est un dépassement de bornes (erreur d'exécution).

EXERCICE RÉSOLU 11 Notes d'une classe de Première D — Max, Min, Moyenne

Énoncé : Un professeur veut saisir les notes d'informatique des n élèves de sa classe (n lu au clavier, $1 \leq n \leq 100$), puis afficher la note maximale, la note minimale et la moyenne de la classe. Écrire l'algorithme complet avec saisie contrôlée de n .

SOLUTION

Raisonnement. On déclare un tableau de taille maximale (constante `MAX = 100`) et on n'utilise que les n premières cases. Une première boucle remplit le tableau en cumulant la somme ; une seconde boucle parcourt le tableau pour trouver le maximum et le minimum, initialisés à $T[1]$.

```

Algorithme NotesClasse
Constante MAX = 100
Variables n, i : Entier
           somme, moyenne, maxi, mini : Reel
Tableau T[1..MAX] : Reel
Debut
  Repeter
    Ecrire("Nombre d'eleves (1 a 100) : ")
    Lire(n)
  Jusqu'a (n >= 1) ET (n <= MAX)

  somme <- 0
  Pour i de 1 a n Faire
    Ecrire("Note de l'eleve ", i, " : ")
    Lire(T[i])
    somme <- somme + T[i]
  FinPour

  maxi <- T[1]
  mini <- T[1]
  Pour i de 2 a n Faire
    Si (T[i] > maxi) Alors
      maxi <- T[i]
    FinSi
    Si (T[i] < mini) Alors
      mini <- T[i]
    FinSi

```

```

FinPour
moyenne <- somme / n
Ecrire("Note maximale : ", maxi)
Ecrire("Note minimale : ", mini)
Ecrire("Moyenne de la classe : ", moyenne)
Fin

```

Test avec $n = 3$ et les notes 12, 7, 16 : somme = $12 + 7 + 16 = 35$; moyenne = $\frac{35}{3} \approx 11,67$; maximum = 16 ; minimum = 7. Conclusion : l'algorithme affiche 16, 7 et environ 11,67. La saisie contrôlée de n par Répéter ... Jusqu'à protège contre tout dépassement des bornes du tableau.

9.5 Fonctions et procédures

DÉFINITION Sous-programme : Fonction vs Procédure

Un **sous-programme** (fonction ou procédure) est un bloc d'instructions nommé, réutilisable, qui structure le programme principal.

- **Fonction** : Calcule et retourne une unique valeur (son type de retour). Elle s'utilise dans une expression : $y \leftarrow \text{Carre}(5)$. Mot-clé : Retourner(valeur).
- **Procédure** : Effectue un traitement (affichage, modification de variables, saisie) sans retourner de valeur. Elle s'appelle comme une instruction : AfficherMenu(). Pas de Retourner.

MÉTHODE Déclarer une fonction ou une procédure

1. Question clé : « Le sous-programme doit-il rendre un seul résultat exploitable dans une expression ? » Oui → Fonction, Non → Procédure.
2. Identifier les paramètres (données d'entrée nécessaires) avec leurs types.
3. Rédiger l'en-tête exact :


```

      | Fonction Carre(x : Reel) : Reel
      | Procédure AfficherSomme(a : Entier, b : Entier)
      
```
4. Corps : déclarations locales éventuelles, instructions, Retourner(...) obligatoire en fin de fonction.
5. Appel : le nombre, l'ordre et les types des arguments doivent correspondre exactement aux paramètres.

EXERCICE RÉSOLU 12 Périmètre et aire d'un terrain rectangulaire

Énoncé : Un promoteur de Yaoundé veut un algorithme qui, à partir de la longueur L et de la largeur l d'une parcelle, calcule son périmètre $P = 2(L + l)$ et son aire $A = L \times l$. Écrire deux fonctions `Perimetre` et `Aire`, puis l'algorithme principal qui les utilise pour $L = 25$ m et $l = 12$ m.

SOLUTION

Raisonnement. Chaque calcul rend un unique résultat réel : ce sont deux fonctions, chacune ayant deux paramètres réels.

```

Fonction Perimetre(L : Reel, l : Reel) : Reel
Debut
    Retourner(2 * (L + l))
Fin

Fonction Aire(L : Reel, l : Reel) : Reel
Debut
    Retourner(L * l)
Fin

Algorithme Parcelle
Variables long, larg, P, A : Reel
Debut
    Ecrire("Longueur et largeur (m) : ")
    Lire(long, larg)
    P <- Perimetre(long, larg) // Appel de fonction dans une affectation
    A <- Aire(long, larg)
    Ecrire("Perimetre = ", P, " m")
    Ecrire("Aire = ", A, " m2")
Fin
  
```

Exécution pour $L = 25$ et $l = 12$:

$$P = 2(L + l) = 2 \times (25 + 12) = 2 \times 37 = 74 \text{ m,}$$

$$A = L \times l = 25 \times 12 = 300 \text{ m}^2.$$

Conclusion : l'algorithme affiche « Perimetre = 74 m » et « Aire = 300 m2 ». Découper en fonctions rend le principal court, lisible et réutilisable.

9.6 Paramètres : passage par valeur et par référence**PROPRIÉTÉ** Modes de passage des paramètres

1. Passage par valeur (mode par défaut) : Le sous-programme reçoit une copie de l'ar-

gument. Toute modification du paramètre à l'intérieur est sans effet sur la variable de l'appelant. À utiliser pour les paramètres entrées (données lues seulement).

2. Passage par référence (ou par variable) : Le sous-programme travaille directement sur la même case mémoire que l'argument (alias). Toute modification est conservée après l'appel. Syntaxe : mot-clé `Var` devant le paramètre. À utiliser quand la procédure doit modifier la variable de l'appelant (échange, saisie, mise à jour, retour de plusieurs résultats).

Règle de décision : « La variable de l'appelant doit-elle être modifiée durablement ? » Oui → `Var` (référence), Non → valeur.

MÉTHODE Identifier le mode de passage

1. Repérer le mot-clé `Var` dans l'en-tête de la procédure/fonction.
2. Si `Var` présent → passage par référence (modification persistante).
3. Si absent → passage par valeur (modification locale, perdue à la fin).
4. Attention : Les tableaux sont toujours passés par référence (adresse de base) en algorithmique standard, même sans `Var` explicite. Une procédure qui modifie un élément `T[i]` modifie le tableau original.

EXERCICE RÉSOLU 13 Échange de deux variables — Effet du mode de passage

Énoncé : On considère la procédure suivante :

```
Procédure Echanger(Var a : Entier, Var b : Entier)
Variable temp : Entier
Debut
    temp ← a
    a ← b
    b ← temp
Fin
```

L'algorithme principal affecte `x ← 5` et `y ← 9`, appelle `Echanger(x, y)`, puis affiche `x` et `y`. (1) Qu'affiche le programme ? Justifier. (2) Qu'afficherait-il si les paramètres étaient passés par valeur (sans `Var`) ?

SOLUTION

(1) Passage par référence (`Var`). `a` et `b` sont des alias de `x` et `y` (mêmes cases mémoire). Déroulé :

- Avant appel : $x = 5, y = 9$.
- `temp <- a` : `temp = 5`.
- `a <- b` : `x` devient 9.
- `b <- temp` : `y` devient 5.

Les modifications persistent. Affichage : $x = 9$ et $y = 5$. L'échange a réussi.

(2) Passage par valeur (sans `var`). La procédure reçoit des copies de x et y . L'échange s'effectue sur les copies locales `a` et `b`, détruites à la fin. x et y du principal sont intacts. Affichage : $x = 5$ et $y = 9$. L'échange a échoué.

Conclusion : dès qu'une procédure doit modifier durablement une variable de l'appelant, `var` (passage par référence) est indispensable.

9.7 Variables locales, variables globales et exécution

DÉFINITION Portée (visibilité) des variables

- **Variable globale** : Déclarée dans l'algorithme principal (section `Variables` avant `Debut`) ou avant tous les sous-programmes. Elle est visible et modifiable partout (principal, fonctions, procédures).
- **Variable locale** : Déclarée à l'intérieur d'une fonction ou procédure (section `Variable locale`). Elle n'existe que pendant l'exécution de ce sous-programme et est invisible à l'extérieur. Deux sous-programmes peuvent avoir des locales de même nom sans conflit.
- **Masquage** : Si une variable locale porte le même nom qu'une globale, c'est la locale qui est utilisée dans le sous-programme (elle « masque » la globale).

Bonne pratique : Réserver les globales aux données réellement partagées (ex. configuration, compteur global). Tout le reste (compteurs de boucle, résultats intermédiaires) doit être local ou passé en paramètre.

MÉTHODE Exécuter un algorithme avec sous-programmes (table d'exécution)

Pour tracer l'exécution pas à pas :

1. Dresser un tableau à trois zones : Variables globales, Paramètres (valeurs reçues), Variables locales.
2. Suivre chronologiquement : exécution du principal → appel → évaluation arguments → exécution corps sous-programme → Retourner → reprise principal.
3. Noter la valeur de chaque variable à chaque modification.

EXERCICE RÉSOLU 14 Exécution avec fonction, variable globale et locale

Énoncé : On considère l'algorithme suivant :

```

Algorithme Test
Variable n : Entier           // Variable globale

Fonction Double(x : Entier) : Entier
Variable r : Entier           // Variable locale
Debut
    r <- 2 * x + n
    Retourner(r)
Fin

Debut
    n <- 1
    Ecrire(Double(3))
    n <- 10
    Ecrire(Double(3))
Fin
  
```

(1) Identifier les variables globales, locales et les paramètres. (2) Déterminer les deux valeurs affichées, en justifiant chaque calcul.

SOLUTION

(1) Classification.

- **n** : déclarée dans le principal, hors fonction → globale (visible dans Double).
- **r** : déclarée dans Double → locale (existe seulement pendant l'appel).
- **x** : paramètre de Double, passé par valeur → reçoit une copie de l'argument.

(2) Exécution pas à pas.

Étape	Globale n	Paramètre x	Locale r	Action / Commentaire
1	1	–	–	n <- 1 (principal)
2	1	3	–	Appel Double(3) : x reçoit 3
3	1	3	7	r <- 2*3 + 1 = 7
4	1	3	7	Retourner(7) → Affichage 1 : 7
5	10	–	–	n <- 10 (principal, globale modifiée)
6	10	3	–	Appel Double(3) : x reçoit 3
7	10	3	16	r <- 2*3 + 10 = 16
8	10	3	16	Retourner(16) → Affichage 2 : 16

Conclusion : le programme affiche 7 puis 16. Le même appel `Double(3)` produit deux résultats différents car la fonction dépend de la globale **n**, modifiée entre les deux appels.

C'est pourquoi l'usage des globales doit rester exceptionnel : il rend le comportement imprévisible et le code difficile à tester.

ATTENTION Les 5 erreurs qui coûtent des points au Probatoire

1. Oublier l'initialisation avant la boucle. Écrire `S <- S + i` sans `S <- 0` avant le `Pour` : la somme part d'une valeur indéfinie (souvent 0 par chance, mais non garanti). Toute boucle de cumul exige une initialisation explicite.
2. Boucle infinie dans un `Tant que`. Aucune variable de la condition n'est modifiée dans le corps (ex. oublier `i <- i + 1`). Toujours vérifier la terminaison sur un exemple simple (trace manuelle).
3. Dépassement des bornes d'un tableau. Pour `T[1..n]`, écrire `T[0]` ou `T[n+1]` ; ou initialiser `max <- 0` au lieu de `max <- T[1]` (faux si toutes valeurs < 0). Indices valides : 1 à n inclus.
4. Confondre fonction et procédure. Fonction sans `Retourner` ou sans type de retour ; procédure utilisée dans une expression (`x <- Proc()`) ; fonction appelée sans exploiter sa valeur (`Carre(5)` seul). Fonction = valeur de retour ; Procédure = effet de bord.
5. Se tromper de mode de passage. Attendre qu'une procédure modifie une variable de l'appelant sans `Var` (passage par valeur) : la modification se perd. Inversement, mettre `Var` sur un paramètre qui ne doit pas être modifié (danger). Justifier toujours le choix de `Var` dans la copie.

10 EXERCICES

10.1 Je vérifie mes connaissances

EXERCICE 8 QCM : structures et sous-programmes

Pour chaque question, une seule réponse est correcte. Recopie la lettre correspondante.

1. La structure itérative dont le nombre de répétitions est connu à l'avance est :
 - a) Si ... Alors ... Sinon
 - b) Tant que ... Faire
 - c) Pour i de 1 à n Faire
 - d) Répéter ... Jusqu'à
2. Un tableau T déclaré Tableau T[1..10] : réel contient :
 - a) 9 éléments
 - b) 10 éléments
 - c) 11 éléments
 - d) un nombre variable d'éléments
3. Une fonction se distingue d'une procédure parce qu'elle :
 - a) ne possède jamais de paramètres
 - b) retourne obligatoirement une valeur
 - c) ne peut pas contenir de boucle
 - d) s'exécute plus rapidement
4. Dans l'en-tête Procédure Calculer(Var x : entier), le paramètre x est passé :
 - a) par valeur
 - b) par référence
 - c) par défaut
 - d) par adresse mémoire absolue

EXERCICE 9 Vrai ou faux ? Justifie

Réponds par Vrai ou Faux et justifie chaque réponse en une ou deux phrases.

1. Une variable déclarée à l'intérieur d'une fonction est utilisable dans l'algorithme principal.
2. La boucle Répéter ... Jusqu'à exécute son corps au moins une fois.
3. En algorithmique, l'indice du premier élément d'un tableau déclaré $T[1..n]$ est 1.
4. Une procédure peut modifier la valeur d'une variable de l'algorithme principal si celle-ci est passée par référence.

EXERCICE 10 Définitions

Définis clairement, en deux ou trois phrases chacune, les termes suivants : **fonction**, **procédure**, **paramètre**, **variable locale**, **variable globale**, **structure itérative**. Pour chacun des deux premiers termes, donne un exemple d'en-tête en écriture algorithmique.

EXERCICE 11 Trace d'exécution rapide

On considère l'algorithme suivant :

```

Algorithme Compte
Variable i, s : entier
Début
    s ← 0
    Pour i de 1 à 5 Faire
        s ← s + i * i
    FinPour
    Écrire(s)
Fin

```

1. Combien de fois le corps de la boucle est-il exécuté ?
2. Donne, sous forme de tableau, la valeur de s à la fin de chaque passage dans la boucle.
3. Quelle valeur est affichée à la fin de l'exécution ?

10.2 Je m'entraîne**EXERCICE 12 Notes d'une classe de Première C**

Le professeur d'informatique du lycée de Bafoussam veut exploiter les notes de sa classe.

1. Écris un algorithme qui lit les notes sur 20 de 20 élèves dans un tableau `Notes`, puis affiche la moyenne de la classe, la note maximale et le nombre d'élèves admis (note ≥ 10).
2. Ajoute une boucle de saisie sécurisée qui impose que chaque note saisie soit comprise entre 0 et 20.

EXERCICE 13 Fonction `EstPremier`

1. Écris une fonction `EstPremier(n : entier) : booléen` qui retourne `Vrai` si l'entier n (avec $n \geq 2$) est premier, et `Faux` sinon.
2. Utilise cette fonction dans un algorithme principal qui affiche tous les nombres premiers strictement inférieurs à 100.
3. Combien de nombres premiers ton algorithme affiche-t-il ? (Tu peux les énumérer à la main pour vérifier.)

EXERCICE 14 Chute libre d'un mobile

Un mobile est lâché depuis une hauteur de 45 m. Sa hauteur restante à l'instant t (en secondes) est donnée par :

$$h(t) = 45 - 4,9 t^2$$

1. Écris un algorithme qui affiche, pour t variant de 0 à 5 s par pas de 0,5 s, la valeur de $h(t)$.
2. Modifie l'algorithme pour qu'il signale, dès que $h(t) \leq 0$, l'instant où le mobile touche le sol, puis s'arrête.
3. Calcule à la main $h(3)$ et vérifie que ton algorithme affiche la même valeur.

EXERCICE 15 Recherche dans un tableau

Le secrétariat d'un collège de Douala a rangé les matricules de 15 élèves dans un tableau `Mat[1..15] : chaîne`.

1. Écris une fonction `EstPresent(Mat : tableau, m : chaîne) : booléen` qui retourne `Vrai` si le matricule m figure dans le tableau.
2. Écris une procédure `AfficherPosition(Mat : tableau, m : chaîne)` qui affiche la position du matricule dans le tableau, ou le message « Matricule introuvable ».

3. Précise, pour chaque sous-programme, la nature de chaque paramètre et le mode de passage le plus adapté, en justifiant.

10.3 Je me prépare au Probatoire

EXERCICE 16 Type Probatoire : la procédure Mystère

On considère la procédure et l'algorithme principal suivants :

Procédure Mystère(a : entier, Var b : entier)

Variable k : entier

Début

k ← -a + b

a ← -a + 10

b ← k * 2

Écrire(a, b)

Fin

Algorithme Principal

Variable x, y : entier

Début

x ← -3

y ← -4

Mystère(x, y)

Écrire(x, y)

Mystère(y, x)

Écrire(x, y)

Fin

1. Identifie, pour chaque paramètre de la procédure `Mystère`, son mode de passage (par valeur ou par référence). Justifie ta réponse à partir de l'en-tête.
2. Dresse la table de trace d'exécution complète de l'algorithme `Principal` : valeurs de x, y, a, b, k après chaque instruction significative.
3. Donne les quatre lignes affichées par le programme, dans l'ordre.
4. Que se passerait-il si l'en-tête était `Procédure Mystère(Var a : entier, Var b : entier)` ? Donne les nouvelles valeurs affichées par la première ligne `Écrire(x, y)`.
5. La variable `k` est-elle locale ou globale ? Justifie.

EXERCICE 17 Type Probatoire : gestion d'un cybercafé

Monsieur Nkoulou gère un cybercafé à Yaoundé. Chaque journée, il enregistre dans un tableau `Rec[1..30]` les recettes journalières (en FCFA) du mois écoulé. Il te demande de l'aider à analyser son activité.

1. Écris une procédure `Saisir(Var Rec : tableau)` qui remplit le tableau avec les recettes des 30 jours, en imposant des valeurs positives ou nulles.
2. Écris une fonction `Total(Rec : tableau) : réel` qui retourne la recette totale du mois.
3. Écris une fonction `JourMax(Rec : tableau) : entier` qui retourne le numéro du jour où la recette a été maximale.
4. Écris une procédure `CompterJoursFaibles(Rec : tableau, seuil : réel, Var nb : entier)` qui compte dans `nb` le nombre de jours où la recette est strictement inférieure à `seuil`. Quel est le mode de passage de `nb`? Justifie.
5. Écris l'algorithme principal qui utilise ces sous-programmes pour afficher : la recette totale, la recette moyenne journalière, le meilleur jour, et le nombre de jours où la recette est restée sous 10000 FCFA.
6. Pour chaque sous-programme écrit, distingue les variables locales des paramètres, et indique le mode de passage de chaque paramètre.

EXERCICE 18 Type Probatoire : tri d'un tableau de moyennes

À la fin du premier trimestre, le conseil de classe de Première D du lycée bilingue d'Ebolowa souhaite classer 5 élèves par ordre décroissant de moyenne. On considère la procédure suivante :

```
Procédure Échanger(Var a : réel, Var b : réel)
```

```
Variable temp : réel
```

```
Début
```

```
    temp <- a
```

```
    a <- b
```

```
    b <- temp
```

```
Fin
```

et le tableau de moyennes : `Moy = [12,5 ; 8,0 ; 15,5 ; 10,0 ; 6,5]`.

1. Explique le rôle de la procédure `Échanger`. Pourquoi ses deux paramètres sont-ils passés par référence? Que se passerait-il s'ils étaient passés par valeur?
2. Écris une procédure `Trier(Var T : tableau)` qui trie le tableau `T` de 5 moyennes par ordre décroissant, en utilisant la méthode du tri par sélection et en faisant appel à la procédure `Échanger`.

3. Exécute à la main ta procédure `Trier` sur le tableau `Moy` : donne l'état du tableau après chaque échange effectué.
4. Écris l'algorithme principal qui saisit les 5 moyennes (comprises entre 0 et 20), appelle `Trier`, puis affiche le mérite de chaque élève : rang 1, rang 2, ..., rang 5.
5. Cite deux autres structures de données que le type tableau, et donne pour chacune un exemple d'utilisation dans la vie courante ou scolaire.

11 CORRIGÉS DES EXERCICES

CORRIGÉ Exercice 1 — QCM : structures de contrôle et sous-programmes

Énoncé : Choisir la bonne réponse parmi les propositions pour chaque question.

1. Réponse c) : Pour i de 1 à n Faire. La boucle Pour s'utilise précisément lorsque le nombre de répétitions est connu avant d'entrer dans la boucle. Si...Alors...Sinon est une structure conditionnelle (pas itérative); Tant que et Répéter...Jusqu'à s'emploient quand le nombre de tours dépend d'une condition évaluée pendant l'exécution.
2. Réponse b) : 10 éléments. Un tableau déclaré $T[1..10]$ possède les indices 1, 2, ..., 10, soit $10 - 1 + 1 = 10$ cases.
3. Réponse b) : retourne obligatoirement une valeur. C'est la différence fondamentale : une **fonction** produit un résultat (mot-clé *Retourner*), alors qu'une **procédure** effectue seulement des actions (affichages, modifications de variables). Les deux peuvent avoir des paramètres et contenir des boucles.
4. Réponse b) : par référence. Le mot-clé *Var* devant le paramètre indique un **passage par référence** : la procédure travaille directement sur la variable de l'appelant, et toute modification du paramètre formel est répercutée sur l'argument effectif.

CORRIGÉ Exercice 2 — Vrai ou faux ? Justifiez

Énoncé : Dire si chaque affirmation est vraie ou fausse et justifier brièvement.

1. Faux. Une variable déclarée à l'intérieur d'une fonction est locale : elle n'existe que pendant l'exécution de cette fonction et est invisible depuis l'algorithme principal. Seules les **variables globales** (déclarées dans l'algorithme principal, avant les sous-programmes) sont accessibles partout.
2. Vrai. Dans la structure Répéter ... Jusqu'à, le test de sortie est placé après le corps de la boucle : celui-ci s'exécute donc au moins une fois, contrairement au Tant que dont le test est en entrée (le corps peut ne jamais s'exécuter si la condition est fausse dès le départ).
3. Vrai. La déclaration $T[1..n]$ fixe explicitement les bornes : le premier indice est 1 et le dernier est n . (Attention : dans certains langages comme le C, le premier indice est 0 ; mais en écriture algorithmique du programme officiel, on respecte strictement les bornes déclarées.)
4. Vrai. Le **passage par référence** (mot-clé *Var*) signifie que le paramètre formel est un

alias de la variable effective de l'appelant : la procédure agit directement sur la case mémoire d'origine, et la modification persiste après le retour de la procédure.

CORRIGÉ Exercice 3 — Définitions des concepts clés

Énoncé : Définir précisément les termes suivants : fonction, procédure, paramètre, variable locale, variable globale, structure itérative.

- **Fonction** : sous-programme qui reçoit éventuellement des paramètres, effectue un calcul et retourne obligatoirement une valeur à l'algorithme appelant via l'instruction `Retourner`. Elle s'utilise dans une expression, comme une fonction mathématique. Exemple d'en-tête : `Fonction Carre(x : réel) : réel.`
- **Procédure** : sous-programme qui effectue une suite d'actions (affichages, saisies, modifications de variables passées par référence) sans retourner de valeur. Elle s'appelle comme une instruction autonome. Exemple d'en-tête : `Procédure AfficherMenu().`
- **Paramètre** : donnée échangée entre l'algorithme appelant et le sous-programme. Le paramètre formel figure dans l'en-tête (avec son type et éventuellement `Var`) ; le paramètre effectif est la valeur ou la variable fournie lors de l'appel.
- **Variable locale** : variable déclarée dans la partie déclarative d'un sous-programme (fonction ou procédure). Elle n'existe que pendant l'exécution de ce sous-programme et n'est accessible nulle part ailleurs (ni dans le principal, ni dans un autre sous-programme).
- **Variable globale** : variable déclarée dans la partie déclarative de l'algorithme principal (avant les sous-programmes). Elle est accessible en lecture et en écriture depuis l'algorithme principal et depuis tous les sous-programmes.
- **Structure itérative** (ou boucle) : structure de contrôle qui permet de répéter un bloc d'instructions. On distingue trois formes canoniques :
 - **Pour** : nombre de tours connu à l'avance (compteur explicite).
 - **Tant que** : test en entrée, corps éventuellement jamais exécuté (boucle pré-conditionnelle).
 - **Répéter ... Jusqu'à** : test en sortie, corps exécuté au moins une fois (boucle post-conditionnelle).

CORRIGÉ Exercice 4 — Trace d'exécution : somme des carrés

Énoncé : Donner le nombre d'itérations, établir la table de trace et la valeur finale affichée

par l'algorithme suivant :

```

Variable i, s : entier
Début
  s <- 0
  Pour i de 1 à 5 Faire
    s <- s + i * i
  FinPour
  Écrire(s)
Fin

```

1. La boucle Pour i de 1 à 5 exécute son corps 5 fois (pour $i = 1, 2, 3, 4, 5$).
2. Table de trace (valeurs à la fin de chaque passage dans le corps de la boucle) :

Passage	i	i * i	s (fin de passage)
1	1	1	1
2	2	4	5
3	3	9	14
4	4	16	30
5	5	25	55

3. La valeur affichée est 55. Vérification : $1^2 + 2^2 + 3^2 + 4^2 + 5^2 = 1 + 4 + 9 + 16 + 25 = 55$.
(Cohérent avec la formule $\sum_{i=1}^5 i^2 = \frac{5 \times 6 \times 11}{6} = 55$.)

CORRIGÉ Exercice 5 — Statistiques sur les notes d'une classe de Première C

Énoncé :

1. Écrire un algorithme qui saisit les 20 notes d'une classe (réelles), puis calcule et affiche la moyenne, la note maximale et le nombre d'élèves ayant la moyenne (≥ 10).
2. Modifier la saisie pour qu'elle rejette toute note hors de l'intervalle $[0 ; 20]$.

1. Algorithme principal (version corrigée et robuste) :

```

Algorithme StatistiquesClasse
Variable Notes : Tableau[1..20] de réel
Variable i, nbAdmis : entier
Variable somme, moyenne, max : réel
Début
  // Saisie des 20 notes
  Pour i de 1 à 20 Faire

```

```

    Écrire("Note de l'élève ", i, " : ")
    Lire(Notes[i])
FinPour

// Initialisation correcte : somme à 0, max sur le premier élément
somme <- 0
max <- Notes[1]
nbAdmis <- 0

// Parcours unique du tableau pour tous les calculs
Pour i de 1 à 20 Faire
    somme <- somme + Notes[i]
    Si Notes[i] > max Alors
        max <- Notes[i]
    FinSi
    Si Notes[i] >= 10 Alors
        nbAdmis <- nbAdmis + 1
    FinSi
FinPour

moyenne <- somme / 20
Écrire("Moyenne de la classe : ", moyenne)
Écrire("Note maximale : ", max)
Écrire("Nombre d'admis : ", nbAdmis)
Fin

```

Remarque de rigueur : Initialiser `somme` avec `Notes[1]` puis boucler de 1 à 20 en ajoutant `Notes[i]` compterait `Notes[1]` deux fois. La méthode sûre est `somme <- 0` puis boucle complète de 1 à n , ou `somme <- Notes[1]` puis boucle de 2 à n .

2. Saisie sécurisée avec Répéter ... Jusqu'à (boucle post-conditionnelle, corps exécuté au moins une fois) :

```

    Pour i de 1 à 20 Faire
        Répéter
            Écrire("Note de l'élève ", i, " (0 à 20) : ")
            Lire(Notes[i])
        Jusqu'à (Notes[i] >= 0) Et (Notes[i] <= 20)
    FinPour

```

Tant que la note saisie est hors de l'intervalle $[0 ; 20]$, la condition de sortie est fausse et la saisie est recommencée. Cela garantit que `Notes[i]` est valide avant de passer à l'élève suivant.

CORRIGÉ Exercice 6 — Fonction EstPremier et nombres premiers < 100

Énoncé :

1. Écrire une fonction `EstPremier(n)` qui retourne `Vrai` si $n \geq 2$ est premier, `Faux` sinon.
2. Écrire l'algorithme principal qui affiche tous les nombres premiers strictement inférieurs à 100.
3. Combien y a-t-il de tels nombres?

1. Un entier $n \geq 2$ est premier s'il n'admet aucun diviseur d tel que $2 \leq d \leq n - 1$ (on peut optimiser en testant jusqu'à $\lfloor \sqrt{n} \rfloor$, mais la version simple est attendue au programme).

```

Fonction EstPremier(n : entier) : booléen
  Variable d : entier
  Début
    Pour d de 2 à n - 1 Faire
      Si n Mod d = 0 Alors
        Retourner Faux // un diviseur trouvé : n n'est pas premier
      FinSi
    FinPour
    Retourner Vrai // aucun diviseur trouvé
  Fin

```

2. Algorithme principal :

```

Algorithme PremiersInferieurs100
  Variable n : entier
  Début
    Pour n de 2 à 99 Faire
      Si EstPremier(n) = Vrai Alors
        Écrire(n)
      FinSi
    FinPour
  Fin

```

3. Les nombres premiers strictement inférieurs à 100 sont : 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97. L'algorithme en affiche donc 25.

CORRIGÉ Exercice 7 — Simulation de la chute libre d'un mobile

Énoncé : Un mobile est lâché sans vitesse initiale d'une hauteur de 45 m. L'équation de la hauteur est $h(t) = 45 - 4,9t^2$ (h en mètres, t en secondes, $g \approx 9,8 \text{ m/s}^2$).

1. Afficher h pour t variant de 0 à 5 s par pas de 0,5 s.
2. Modifier l'algorithme pour qu'il s'arrête dès que le mobile touche le sol ($h \leq 0$) et affiche l'instant du contact.
3. Vérifier à la main la valeur affichée pour $t = 3 \text{ s}$.

1. Version simple avec Tant que (boucle pré-conditionnelle) :

```

Algorithme ChuteLibre
Variable t, h : réel
Début
    t ← 0
    Tant que t ≤ 5 Faire
        h ← 45 - 4,9 * t * t
        Écrire("t = ", t, " s   h = ", h, " m")
        t ← t + 0,5
    FinTantQue
Fin

```

2. Version avec détection du contact au sol (condition d'arrêt ajoutée dans le Tant que) :

```

Algorithme ChuteLibreSol
Variable t, h : réel
Début
    t ← 0
    h ← 45
    Tant que (h > 0) Et (t ≤ 5) Faire
        h ← 45 - 4,9 * t * t
        Si h ≤ 0 Alors
            Écrire("Le mobile touche le sol à t = ", t, " s")
        Sinon
            Écrire("t = ", t, " s   h = ", h, " m")
        FinSi
        t ← t + 0,5
    FinTantQue
Fin

```

3. Calcul manuel pour $t = 3$ s :

$$h(3) = 45 - 4,9 \times 3^2 = 45 - 4,9 \times 9 = 45 - 44,1 = 0,9 \text{ m.}$$

L'algorithme affiche bien $h = 0,9$ m pour $t = 3$ s. Au pas suivant, $t = 3,5$: $h(3,5) = 45 - 4,9 \times 12,25 = 45 - 60,025 = -15,025 \leq 0$. La version 2 signale donc que le mobile touche le sol à $t = 3,5$ s (valeur approchée par excès due au pas de 0,5 s ; l'instant exact est $t = \sqrt{45/4,9} \approx 3,03$ s).

CORRIGÉ

Exercice 8 — Recherche séquentielle dans un tableau de chaînes

Énoncé : Soit un tableau $\text{Mat}[1..15]$ de chaînes (matricules).

1. Écrire une fonction `EstPresent` qui retourne `Vrai` si une chaîne m est dans le tableau.
2. Écrire une procédure `AfficherPosition` qui affiche l'indice de m s'il existe, sinon "In-

trouvable”.

3. Préciser la nature et le mode de passage de chaque paramètre.

1. Fonction de recherche (retourne un booléen, arrêt précoce dès la découverte) :

Fonction EstPresent(Mat : Tableau[1..15] de chaîne, m : chaîne) : booléen

Variable i : entier

Début

Pour i de 1 à 15 Faire

Si Mat[i] = m Alors

Retourner Vrai

FinSi

FinPour

Retourner Faux

Fin

2. Procédure d’affichage de la position (utilise un drapeau `trouve` et une boucle `Tant que`) :

Procédure AfficherPosition(Mat : Tableau[1..15] de chaîne, m : chaîne)

Variable i : entier

Variable trouve : booléen

Début

trouve ← Faux

i ← 1

Tant que (i ≤ 15) Et (trouve = Faux) Faire

Si Mat[i] = m Alors

trouve ← Vrai

Sinon

i ← i + 1

FinSi

FinTantQue

Si trouve = Vrai Alors

Écrire("Matricule trouvé à la position ", i)

Sinon

Écrire("Matricule introuvable")

FinSi

Fin

3. Analyse des paramètres :

- **Mat** : paramètre donnée (entrée seule), type tableau de chaînes. Passé par valeur (absence de `Var`) : les sous-programmes ne font que lire le tableau, ils ne le modifient pas. Le passage par valeur protège le tableau original (même si pour un tableau cela implique une copie conceptuelle, la sémantique "entrée seule" est respectée).
- **m** : paramètre donnée (entrée seule), type chaîne. Passé par valeur : c'est la valeur recherchée, fournie en entrée et non modifiée.

- Pour `EstPresent`, le résultat est rendu par la valeur de retour (booléen); pour `AfficherPosition`, le résultat est produit par un effet de bord (affichage), d'où le choix d'une procédure sans paramètre résultat.

CORRIGÉ Exercice 9 — Type Probatoire : la procédure Mystère (passage mixte)

Énoncé : On considère la procédure suivante :

```

Procédure Mystère(a : entier, Var b : entier)
Variable k : entier
Début
  k ← - a + b
  a ← - a + 10
  b ← - k * 2
  Écrire(a, b)
Fin

```

Et l'algorithme principal :

```

Algorithme Principal
Variable x, y : entier
Début
  x ← - 3
  y ← - 4
  Mystère(x, y)
  Écrire(x, y)
  Mystère(y, x)
  Écrire(x, y)
Fin

```

1. Préciser le mode de passage de `a` et `b`. Justifier.
2. Établir la table de trace complète (variables `x`, `y`, `a`, `b`, `k`).
3. Donner les quatre lignes affichées dans l'ordre.
4. Que se passerait-il si l'en-tête était `Procédure Mystère(Var a : entier, Var b : entier)` ? Donner la première ligne affichée par `Écrire(x, y)`.
5. `k` est-elle locale ou globale ? Justifier.

1. Dans l'en-tête `Procédure Mystère(a : entier, Var b : entier)` :

- `a` est passé par valeur (absence du mot-clé `Var`) : la procédure travaille sur une copie de la variable effective ; les modifications de `a` ne sont pas répercutées sur l'argument de l'appelant.

- **b est passé par référence (présence de Var) : b est un alias de la variable effective ; toute modification de b modifie directement la variable de l'appelant.**

2. Table de trace complète :

Instruction	x	y	a	b	k
<code>x <- 3</code>	3	-			
<code>y <- 4</code>	3	4			
Appel <code>Mystère(x, y) : k <- a + b</code>	3	4	3	4	7
<code>a <- a + 10</code>	3	4	13	4	7
<code>b <- k * 2</code>	3	14	13	14	7
<code>Écrire(a, b)</code>	3	14	13	14	7
Retour , <code>Écrire(x, y)</code>	3	14			
Appel <code>Mystère(y, x) : k <- a + b</code>	3	14	14	3	17
<code>a <- a + 10</code>	3	14	24	3	17
<code>b <- k * 2</code>	34	14	24	34	17
<code>Écrire(a, b)</code>	34	14	24	34	17
Retour , <code>Écrire(x, y)</code>	34	14			

Point clé : au premier appel, b est lié à y (donc y devient 14), mais a n'est qu'une copie de x (donc x reste 3). Au second appel `Mystère(y, x)`, l'ordre des arguments est inversé : a reçoit la valeur de y (14, par valeur), b est lié à x (par référence) : x devient 34, tandis que y (copié dans a) reste 14.

3. Les quatre lignes affichées, dans l'ordre :

```
13 14
3 14
24 34
34 14
```

4. Si l'en-tête était `Procédure Mystère(Var a : entier, Var b : entier)`, alors a serait aussi passé par référence : lors du premier appel `Mystère(x, y)`, a serait lié à x, donc x prendrait la valeur $3 + 10 = 13$. La première ligne `Écrire(x, y)` afficherait alors : 13 14.

5. La variable k est locale : elle est déclarée à l'intérieur de la procédure `Mystère`, elle n'existe que pendant son exécution et est invisible depuis l'algorithme `Principal`.

Barème indicatif (sur 10 pts) : modes de passage justifiés : 2 pts ; table de trace exacte : 3 pts ; quatre lignes affichées : 2 pts ; cas des deux passages par référence : 2 pts ; variable locale justifiée : 1 pt.

Points de vigilance : exiger la justification du mode de passage à partir du mot-clé *Var* ; vérifier que l'élève comprend qu'au second appel, l'ordre des arguments est inversé (y puis x), ce qui change la variable modifiée ; le passage par valeur protège la variable de l'appelant — citer cette condition d'application.

CORRIGÉ**Exercice 10 — Type Probatoire : gestion d'un cybercafé (tableau, sous-programmes)**

Énoncé : On souhaite gérer les recettes journalières (en FCFA) d'un cybercafé sur un mois de 30 jours. Le tableau `Rec[1..30]` stocke ces recettes.

1. Écrire une procédure `Saisir` qui remplit le tableau avec contrôle de validité (recette ≥ 0).
2. Écrire une fonction `Total` qui retourne la somme des recettes.
3. Écrire une fonction `JourMax` qui retourne le numéro du jour où la recette a été maximale.
4. Écrire une procédure `CompterJoursFaibles` qui compte le nombre de jours avec recette $< 10\,000$ FCFA.
5. Écrire l'algorithme principal qui utilise ces sous-programmes et affiche : total, moyenne, meilleur jour, nombre de jours faibles.
6. Dresser le bilan (tableau) des sous-programmes : paramètres (mode), variables locales, résultat.

1. Procédure de saisie sécurisée (tableau passé par référence pour être modifié) :

```

Procédure Saisir(Var Rec : Tableau[1..30] de réel)
Variable i : entier
Début
    Pour i de 1 à 30 Faire
        Répéter
            Écrire("Recette du jour ", i, " (FCFA) : ")
            Lire(Rec[i])
        Jusqu'à Rec[i] >= 0
    FinPour
Fin
  
```

2. Fonction de calcul du total (tableau passé par valeur car lecture seule) :

```

Fonction Total(Rec : Tableau[1..30] de réel) : réel
Variable i : entier
Variable s : réel
Début
    s <- 0
    Pour i de 1 à 30 Faire
        s <- s + Rec[i]
    FinPour
    Retourner s
Fin
  
```

3. Fonction de recherche du meilleur jour (retourne l'indice, pas la valeur) :

```

Fonction JourMax(Rec : Tableau[1..30] de réel) : entier
  
```

```
Variable i, jMax : entier
Début
  jMax ← - 1
  Pour i de 2 à 30 Faire
    Si Rec[i] > Rec[jMax] Alors
      jMax ← i
    FinSi
  FinPour
  Retourner jMax
Fin
```

4. Procédure de comptage (paramètre `nb` passé par référence car paramètre résultat) :

```
Procédure CompterJoursFaibles(Rec : Tableau[1..30] de réel,
                             seuil : réel, Var nb : entier)
Variable i : entier
Début
  nb ← 0
  Pour i de 1 à 30 Faire
    Si Rec[i] < seuil Alors
      nb ← nb + 1
    FinSi
  FinPour
Fin
```

Le paramètre `nb` est passé par référence (mot-clé `Var`) : c'est un paramètre résultat (ou sortie). La procédure ne retournant rien, c'est par `nb` que le comptage est transmis à l'algorithme appelant. Sans `Var`, la valeur calculée dans la copie locale serait perdue au retour.

5. Algorithme principal :

```
Algorithme Cybercafe
Variable Rec : Tableau[1..30] de réel
Variable total, moyenne : réel
Variable meilleur, nbFaibles : entier
Début
  Saisir(Rec)
  total ← Total(Rec)
  moyenne ← total / 30
  meilleur ← JourMax(Rec)
  CompterJoursFaibles(Rec, 10000, nbFaibles)
  Écrire("Recette totale du mois : ", total, " FCFA")
  Écrire("Recette moyenne journalière : ", moyenne, " FCFA")
  Écrire("Meilleur jour : jour ", meilleur)
  Écrire("Jours sous 10000 FCFA : ", nbFaibles)
Fin
```

6. Bilan de synthèse :

Sous-programme	Paramètres (mode)	Variables locales	Résultat
Saisir	Rec (par référence)	i	tableau rempli (effet de bord)
Total	Rec (par valeur)	i, s	valeur retournée (réel)
JourMax	Rec (par valeur)	i, jMax	valeur retournée (entier)
CompterJoursFaibles	Rec (par valeur), seuil (par valeur), nb (par référence)	i	via nb (entier)

Barème indicatif (sur 12 pts) : Saisir avec contrôle de validité : 2 pts ; Total : 2 pts ; JourMax : 2 pts ; CompterJoursFaibles + justification du passage par référence de nb : 2 pts ; algorithme principal cohérent (ordre des appels, calcul de la moyenne) : 2 pts ; tableau de synthèse local/paramètre et modes de passage : 2 pts.

Points de vigilance : le tableau Rec doit être passé par référence dans Saisir (sinon le remplissage est perdu) mais par valeur dans les fonctions de lecture seule ; ne pas confondre le numéro du jour (indice) avec la recette maximale ; la moyenne se calcule après l'appel à Total, jamais en additionnant des moyennes partielles ; penser à initialiser `nb <- 0` dans la procédure.

CORRIGÉ Exercice 11 — Type Probatoire : tri par sélection d'un tableau de moyennes

Énoncé : On dispose d'un tableau `Moy[1..5]` de réels (moyennes d'élèves).

1. Expliquer le rôle d'une procédure `Échanger(x, y)` et justifier le mode de passage de ses paramètres.
2. Écrire une procédure `Trier` qui trie le tableau par ordre décroissant en utilisant `Échanger` (tri par sélection).
3. Effectuer la trace manuelle sur `Moy = [12,5 ; 8,0 ; 15,5 ; 10,0 ; 6,5]`.
4. Écrire l'algorithme principal : saisie contrôlée (0 à 20), tri, affichage des rangs.
5. Citer deux autres structures de données vues au cours avec un exemple concret pour chacune.

1. La procédure `Échanger` permute les contenus de deux variables réelles grâce à une variable intermédiaire `temp`. Ses deux paramètres doivent être passés par référence (mot-clé `Var`) car la procédure doit modifier durablement les deux variables de l'appelant. S'ils étaient passés par valeur, la procédure échangerait seulement des copies : à la sortie, les

variables d'origine conserveraient leurs valeurs initiales et l'échange serait sans effet visible à l'extérieur.

```
Procédure Échanger(Var x : réel, Var y : réel)
```

```
Variable temp : réel
```

```
Début
```

```
temp <- x
```

```
x <- y
```

```
y <- temp
```

```
Fin
```

2. Tri par sélection décroissant (on place le maximum à chaque tour) :

```
Procédure Trier(Var T : Tableau[1..5] de réel)
```

```
Variable i, j, iMax : entier
```

```
Début
```

```
Pour i de 1 à 4 Faire // n-1 tours suffisent
```

```
iMax <- i
```

```
Pour j de i + 1 à 5 Faire // recherche du max dans la partie non triée
```

```
Si T[j] > T[iMax] Alors
```

```
iMax <- j
```

```
FinSi
```

```
FinPour
```

```
Si iMax <> i Alors // échange seulement si nécessaire
```

```
Échanger(T[i], T[iMax])
```

```
FinSi
```

```
FinPour
```

```
Fin
```

3. Exécution à la main sur Moy = [12,5 ; 8,0 ; 15,5 ; 10,0 ; 6,5] :

Étape	Action	État du tableau
Départ	–	[12,5 ; 8,0 ; 15,5 ; 10,0 ; 6,5]
$i = 1$	max = 15,5 en position 3; échange $T[1] \leftrightarrow T[3]$	[15,5 ; 8,0 ; 12,5 ; 10,0 ; 6,5]
$i = 2$	max = 12,5 en position 3; échange $T[2] \leftrightarrow T[3]$	[15,5 ; 12,5 ; 8,0 ; 10,0 ; 6,5]
$i = 3$	max = 10,0 en position 4; échange $T[3] \leftrightarrow T[4]$	[15,5 ; 12,5 ; 10,0 ; 8,0 ; 6,5]
$i = 4$	max = 8,0 en position 4; aucun échange ($iMax = i$)	[15,5 ; 12,5 ; 10,0 ; 8,0 ; 6,5]

Le tableau final est bien trié par ordre décroissant : $15,5 \geq 12,5 \geq 10,0 \geq 8,0 \geq 6,5$.

4. Algorithme principal :

```
Algorithme TableauMerite
```

```
Variable Moy : Tableau[1..5] de réel
```

```

Variable i : entier
Début
  Pour i de 1 à 5 Faire
    Répéter
      Écrire("Moyenne de l'élève ", i, " (0 à 20) : ")
      Lire(Moy[i])
    Jusqu'à (Moy[i] >= 0) Et (Moy[i] <= 20)
  FinPour
  Trier(Moy)
  Pour i de 1 à 5 Faire
    Écrire("Rang ", i, " : ", Moy[i])
  FinPour
Fin

```

5. Deux autres structures de données fondamentales :

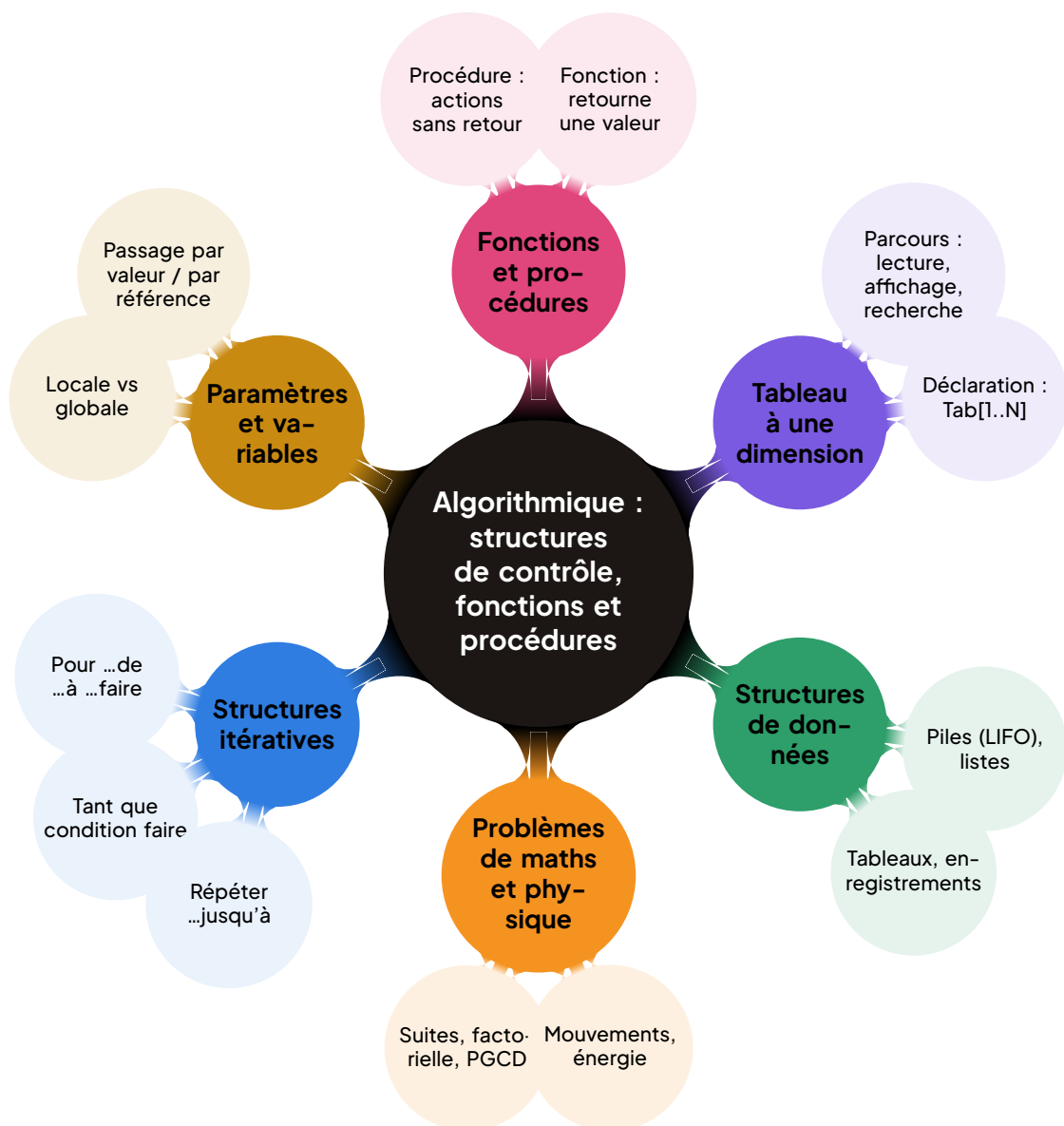
- L'**enregistrement** (ou structure) : regroupe des champs de types différents décrivant un même objet complexe. Exemple : la fiche d'un élève dans le fichier du conseil de classe (Nom : chaîne, Âge : entier, Moyenne : réel, Série : chaîne).
- La **pile** (Stack) : structure LIFO (Last In, First Out — dernier arrivé, premier sorti). Deux opérations de base : Empiler (ajouter au sommet) et Dépiler (retirer du sommet). Exemple : l'historique « Annuler » (Ctrl+Z) d'un éditeur de texte — on ne peut annuler que la dernière action effectuée ; ou la pile de cahiers sur le bureau du professeur.

(On pouvait aussi citer la liste : suite ordonnée d'éléments de même type, taille variable, ex. liste d'attente à la cantine, liste des inscrits au club informatique.)

Barème indicatif (sur 12 pts) : rôle d'Échanger et justification du passage par référence (avec contre-exemple par valeur) : 3 pts ; procédure Trier correcte (double boucle, indice du maximum, appel à Échanger) : 3 pts ; trace manuelle exacte après chaque échange : 2 pts ; algorithme principal (saisie contrôlée, appel, affichage des rangs) : 2 pts ; deux structures de données avec exemples concrets : 2 pts.

Points de vigilance : exiger la justification « passage par référence car modification durable des variables de l'appelant » ; dans le tri par sélection, la boucle externe s'arrête à $n-1$ (ici 4) et non à n ; ne pas oublier le test Si $i_{\text{Max}} <> i$ pour éviter un échange inutile d'un élément avec lui-même ; pour un tri décroissant, on cherche le maximum (comparaison $>$), pas le minimum.

12 FICHE BILAN



Carte mentale de la leçon : les six compétences clés à maîtriser.

FICHE BILAN L'essentiel à connaître pour l'examen

1. Définitions clés

- **Structure itérative** : structure de contrôle qui répète un bloc d'instructions tant qu'une condition est vérifiée (ou un nombre de fois connu).

- **Tableau** : structure de données regroupant sous un même nom plusieurs éléments de même type, accessibles par un **indice**.
- **Fonction** : sous-algorithme qui reçoit des paramètres et retourne une valeur (utilisable dans une expression).
- **Procédure** : sous-algorithme qui effectue des actions sans retourner de valeur.
- **Paramètre** : donnée transmise à une fonction ou procédure ; **variable locale** : déclarée et visible uniquement dans le sous-algorithme ; **variable globale** : déclarée dans l'algorithme principal, visible partout.

2. Syntaxes à connaître par cœur

Notion	Syntaxe algorithmique
Boucle Pour (nombre d'itérations connu)	Pour i de 1 à N faire ... FinPour
Boucle Tant que (test en entrée)	Tant que (condition) faire ... FinTantque
Boucle Répéter (test en sortie, ≥ 1 passage)	Répéter ... Jusqu'à (condition)
Déclaration d'un tableau	Variable T : Tableau[1..N] de Réel
Déclaration d'une fonction	Fonction carre(x : Réel) : Réel
Déclaration d'une procédure	Procédure afficher(T : Tableau, N : Entier)

3. Méthodes express

- Choisir la bonne boucle : nombre de répétitions connu $\Rightarrow$ Pour ; condition d'arrêt, 0 passage possible $\Rightarrow$ Tant que ; au moins un passage obligatoire $\Rightarrow$ Répéter.
- Parcours d'un tableau : toujours une boucle Pour i de 1 à N ; initialiser un compteur ou un maximum avant la boucle.
- Recherche dans un tableau : parcourir et comparer chaque élément à la valeur cherchée ; mémoriser l'indice trouvé (ou -1 si absent).
- Fonction ou procédure ? : un résultat à réutiliser dans un calcul $\Rightarrow$ fonction ; un affichage ou une modification $\Rightarrow$ procédure.
- Passage de paramètres : par valeur = copie, l'originale est inchangée ; par référence (variable) = la variable d'origine peut être modifiée.
- Exécuter un algorithme : construire une table d'exécution (une colonne par variable, une ligne par itération).

À RETENIR

Checklist avant le Bac

- ☐ Je sais distinguer les trois boucles (Pour, Tant que, Répéter) et choisir la bonne selon le problème.
- ☐ Je vérifie toujours que ma boucle se termine (condition d'arrêt atteignable, compteur incrémenté).
- ☐ Je sais écrire un algorithme de calcul classique : factorielle, somme des n premiers entiers, PGCD, terme d'une suite.
- ☐ Je sais énumérer quatre structures de données : tableaux, enregistrements, piles (LIFO), listes.
- ☐ Je sais déclarer un tableau à une dimension avec son type et sa taille.
- ☐ Je sais parcourir un tableau : lecture au clavier, affichage, recherche d'un élément, calcul de somme ou de maximum.
- ☐ Je respecte les bornes du tableau : indices de 1 à N , jamais 0 ni $N + 1$ en algorithmique MINESEC.
- ☐ Je sais distinguer une fonction (retourne une valeur) d'une procédure (effectue des actions).
- ☐ Je sais identifier le mode de passage d'un paramètre : par valeur ou par référence.
- ☐ Je sais distinguer une variable locale d'une variable globale et prévoir leur portée.
- ☐ Je sais exécuter pas à pas un algorithme avec fonctions et procédures grâce à une table d'exécution.
- ☐ Je relis mon algorithme : chaque variable est déclarée, initialisée, et chaque Si/Pour/Tant que est bien fermé.

**Signé • OnBuch+**

Ludovic A.

CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+